FIAP - Faculdade de Informática e Administração Paulista

Pós-Graduação em Inteligência Artificial para Devs

ASSISTENTE VIRTUAL

MÉDICO COM IA GENERATIVA

Fine-Tuning de LLM, RAG com Protocolos Clínicos e Grafo Multi-Agente com LangGraph

TECH CHALLENGE - FASE 3

Grupo "Sala 14"

Adriana Martins de Souza - RM 368050
Diego Oliveira da Silva - RM 367964
Eduardo Nicola F. Zagari - RM 368021
Renan de Assis Torres - RM 368513

São Paulo

Março de 2026

Grupo "Sala 14"

ASSISTENTE VIRTUAL MÉDICO COM IA GENERATIVA

Fine-Tuning de LLM com MedQuAD, RAG com Protocolos Clínicos e Orquestração Multi-Agente com LangGraph

Relatório técnico apresentado como parte dos requisitos para aprovação no Tech Challenge da Fase 3 do curso de Pós-Graduação em Inteligência Artificial para Devs da FIAP.

São Paulo

Março de 2026

Disponibilização do Código e Materiais

Todo o código desenvolvido neste projeto, bem como esta documentação e o link para o vídeo demonstrativo estão disponíveis publicamente em:

GitHub: https://github.com/Zagari/virtual-medical-assistant

Conteúdo Disponível:

- Pipeline completo de reprodução com 7 scripts numerados (`scripts/01` a `scripts/07`)
- Código-fonte do fine-tuning (QLoRA) e notebook para Google Colab
- 14 Linhas de Cuidado + 5 protocolos de emergência (IAM, AVC, Sepse, Pneumonia, ITU) para RAG
- Dados sintéticos de 80 pacientes com exames, prontuários e receitas
- Grafo multi-agente LangGraph com 7 agentes (6 especializados + logger de auditoria)
- Este relatório técnico em PDF

Vídeo demonstrativo: https://youtube.com/TODO

Licença: MIT License (código aberto para fins educacionais e de pesquisa)

Resumo

Este documento apresenta a documentação técnica completa do Assistente Virtual Médico desenvolvido como parte do Tech Challenge da Fase 3 do curso de Pós-Graduação em Inteligência Artificial para Devs da FIAP. O sistema integra cinco pilares de Inteligência Artificial (IA) generativa: (1) **fine-tuning** do modelo Mistral 7B Instruct v0.3 com o dataset MedQuAD traduzido para português, utilizando Quantized Low-Rank Adaptation (QLoRA) 4-bit no Google Colab; (2) **RAG** (*Retrieval-Augmented Generation*) com 14 Linhas de Cuidado reais e 5 protocolos de emergência sintéticos indexados no ChromaDB; (3) **grafo multi-agente** em LangGraph com 7 agentes — 6 especializados (triagem, protocolo, paciente, raciocínio, guardrails e explicabilidade) e 1 de infraestrutura (logger de auditoria); (4) **segurança** com guardrails que impedem prescrições diretas e exigem validação humana; e (5) **interface** Gradio para demonstração interativa.

O projeto foi concebido para ser integralmente reprodutível: um pipeline de 7 scripts numerados permite que se reconstrua todo o sistema do zero, desde a geração de dados sintéticos até a execução do assistente completo. Os dados de pacientes são inteiramente fictícios, gerados com coerência clínica para simular cenários reais de atendimento hospitalar.

Palavras-chave: Fine-Tuning; QLoRA; Mistral 7B; LangChain; LangGraph; RAG; ChromaDB; Multi-Agente; Guardrails; Assistente Médico.

Abstract

This document presents the complete technical documentation of the Virtual Medical Assistant developed as part of the Phase 3 Tech Challenge of the Postgraduate course in Artificial Intelligence for Developers at FIAP. The system integrates five pillars of generative AI: (1) **fine-tuning** of Mistral 7B Instruct v0.3 with the MedQuAD dataset translated to Portuguese, using QLoRA 4-bit on Google Colab; (2) **RAG** (Retrieval-Augmented Generation) with 14 real clinical care pathways and 5 synthetic emergency protocols indexed in ChromaDB; (3) **multi-agent graph** in LangGraph with 7 agents — 6 specialized (triage, protocol, patient data, reasoning, guardrails, and explainability) and 1 infrastructure agent (audit logger); (4) **safety** guardrails preventing direct prescriptions and requiring human validation; and (5) **Gradio interface** for interactive demonstration.

The project was designed to be fully reproducible: a pipeline of 7 numbered scripts allows the entire system to be rebuilt from scratch, from synthetic data generation to running the complete assistant. Patient data is entirely fictitious, generated with clinical coherence to simulate real hospital care scenarios.

Keywords: Fine-Tuning; QLoRA; Mistral 7B; LangChain; LangGraph; RAG; ChromaDB; Multi-Agent; Guardrails; Medical Assistant.

Sumário

Disponibilização do Código e Materiais i

Resumo ii

Abstract iii

Lista de Abreviaturas e Siglas viii

1 Introdução 1

1.1 Contexto e Motivação 1

1.2 Objetivos do Projeto 1

1.3 Arquitetura Geral do Sistema 2

1.4 Quickstart: Testando em 5 Minutos 3

1.4.1 Pré-requisitos 3

1.4.2 Execução 3

1.5 Guia de Reprodução Passo a Passo 5

1.5.1 Pipeline de Scripts 5

1.5.2 Fluxo de Execução 7

1.5.3 Como Reproduzir 7

1.6 Estrutura de Arquivos do Projeto 9

1.7 Organização do Documento 10

2 Dados e Preparação 11

2.1 Linhas de Cuidado (Protocolos Clínicos) 11

2.1.1 Protocolos Disponíveis 11

2.1.2 Extração e Limpeza dos Protocolos 12

2.1.3 Protocolos Complementares de Emergência 13

2.2 Dados Sintéticos de Pacientes 13

2.2.1 Geração de Pacientes 14

2.2.2 Coerência Clínica 14

2.2.3 Dados Clínicos Gerados 15

2.3 Banco de Dados PostgreSQL 15

2.3.1 Schema do Banco 15
2.3.2 Características Técnicas 17
2.3.3 Infraestrutura Docker 17
2.4 Dataset MedQuAD 17
2.4.1 Escolha da Versão e Limitações de Copyright 18
2.4.2 Características do Dataset 18
2.4.3 Abordagem Híbrida de Treinamento 19
2.4.4 QA de Medicina Brasileira 19
2.4.5 Pipeline de Preparação 20
2.4.6 Composição Final do Dataset 20

3 Fine-Tuning do Modelo 22
3.1 Modelo Base: Mistral 7B Instruct 22
3.1.1 Justificativa da Escolha 22
3.1.2 Características do Mistral 7B 22
3.2 Dataset: MedQuAD e Abordagem Híbrida 23
3.2.1 Escolha do Dataset 23
3.2.2 Fontes do MedQuAD 24
3.2.3 Abordagem Híbrida: PT + EN + Brasileiro 24
3.2.4 Tradução para Português 24
3.2.5 Formato Alpaca 27
3.3 Técnica: QLoRA 28
3.3.1 Fundamentos do LoRA 28
3.3.2 Quantização 4-bit 28
3.3.3 Configuração do LoRA 29
3.4 Processo de Treino 30
3.4.1 Ambiente de Execução 30
3.4.2 Hiperparâmetros de Treinamento 30
3.4.3 Sistema de Checkpoints para Colab Free 32
3.4.4 Execução Local 33
3.4.5 Resultados do Treinamento 33
3.5 Persistência do Modelo 34
3.5.1 Salvando os Adapters LoRA 34
3.5.2 Publicação no Hugging Face Hub 35
3.5.3 Carregando o Modelo para Uso 35
3.5.4 Exportação para GGUF (Inferência sem GPU) 36
3.6 Avaliação do Fine-Tuning 36
3.6.1 Metodologia 36
3.6.2 Conjunto de Perguntas de Avaliação 37

3.6.3 Métricas de Avaliação 37
3.6.4 Scripts de Avaliação 37
3.6.5 Resultados Esperados 38
3.7 Resumo do Capítulo 38

4 RAG: Retrieval-Augmented Generation 40
4.1 Motivação e Arquitetura 40
4.1.1 Componentes do Sistema 40
4.2 Modelo de Embeddings 41
4.2.1 Análise de Alternativas 41
4.2.2 Modelo Selecionado: E5 Multilingual 41
4.3 Estratégia de Chunking 42
4.3.1 Abordagens Consideradas 42
4.3.2 Implementação: Chunking Semântico 42
4.3.3 Exemplo de Chunking 43
4.4 Indexação no ChromaDB 44
4.4.1 Processo de Indexação 44
4.4.2 Estrutura dos Metadados 45
4.4.3 Estatísticas da Indexação 45
4.5 Recuperação de Contexto 45
4.5.1 Processo de Busca 46
4.5.2 Exemplo de Recuperação 46
4.5.3 Formatação do Contexto 47
4.6 Integração com o LLM 47
4.7 Validação do Sistema RAG 48
4.7.1 Métricas de Qualidade 48
4.7.2 Resultados dos Testes 49
4.8 Considerações de Desempenho 49
4.8.1 Tempo de Indexação 49
4.8.2 Tempo de Busca 49

5 Grafo Multi-Agente com LangGraph 51
5.1 Motivação 51
5.2 Visão Geral da Arquitetura 51
5.2.1 Fluxos Possíveis 52
5.3 State Schema 52
5.4 Os 7 Nós do Grafo 54
5.4.1 Triagem (Router) 54
5.4.2 Protocolo (RAG ChromaDB) 55
5.4.3 Paciente Data (PostgreSQL) 56

5.4.4 Raciocínio (Mistral 7B Fine-Tuned) 56
5.4.5 Guardrails (Segurança) 57
5.4.6 Explicabilidade (Citações e Confiança) 58
5.4.7 Logger (Auditoria) 59
5.5 Montagem do Grafo 60
5.6 Funções de Roteamento 61
5.7 Exemplo de Execução End-to-End 62
5.8 Estrutura de Arquivos 63
5.9 Decisões de Design 63
5.10 Interface Gradio 64

6 Segurança, Guardrails e Explicabilidade 65
6.1 Princípios de Segurança em IA Médica 65
6.2 Guardrails: Validação Automática de Respostas 65
6.2.1 Regra 1: Proibição de Prescrição Direta 66
6.2.2 Regra 2: Validação Humana Obrigatória 66
6.2.3 Regra 3: Proibição de Diagnóstico Definitivo 66
6.2.4 Regra 4: Consistência com Contexto 67
6.2.5 Execução Sequencial das Regras 67
6.2.6 Mecanismo de Retry com Feedback 68
6.3 Explicabilidade 68
6.3.1 Citações e Rastreabilidade 68
6.3.2 Avaliação de Confiança 69
6.3.3 Advertências Contextuais 69
6.3.4 Disclaimer Obrigatório 70
6.3.5 Formato da Resposta Final 70
6.4 Auditoria e Logging 70
6.4.1 Estrutura do Registro de Auditoria 70
6.4.2 Persistência em JSONL 71
6.4.3 Rastreabilidade End-to-End 71
6.5 Tratamento de Queries Fora de Escopo 72
6.6 Limitações e Trabalhos Futuros 72

Lista de Abreviaturas e Siglas

CID Classificação Internacional de Doenças. 13

IA Inteligência Artificial. ii, 1, 56

LLM Large Language Model. 1, 22

LoRA Low-Rank Adaptation. 23

QLoRA Quantized Low-Rank Adaptation. ii, 1

RAG Retrieval-Augmented Generation. 2, 33

1

Introdução

1.1 Contexto e Motivação

A Inteligência Artificial (IA) generativa tem transformado diversas áreas do conhecimento, e o setor de saúde desponta como um dos campos com maior potencial de impacto. A capacidade dos Large Language Model (LLM) de processar, interpretar e gerar texto em linguagem natural abre oportunidades significativas para auxiliar profissionais de saúde em tarefas como consulta a protocolos clínicos, suporte à tomada de decisão e acesso rápido a informações de pacientes.

No entanto, modelos generalistas de linguagem — apesar de seu amplo conhecimento — apresentam limitações importantes no domínio médico: podem gerar informações imprecisas, desatualizadas ou descontextualizadas. Essas limitações tornam necessário o uso de técnicas especializadas como **fine-tuning** (ajuste fino) para aprimorar o conhecimento médico do modelo, **RAG** (*Retrieval-Augmented Generation*) para fundamentar respostas em protocolos clínicos reais, e **guardrails** de segurança para garantir que o sistema nunca substitua o julgamento clínico humano.

Este projeto foi desenvolvido no contexto do **Tech Challenge da Fase 3** do curso de Pós-Graduação em Inteligência Artificial para Devs da FIAP, cujo tema central é **IA Generativa**. O desafio propõe a construção de um assistente virtual inteligente que integre técnicas de fine-tuning de LLMs, orquestração multi-agente com LangGraph e segurança por design.

1.2 Objetivos do Projeto

O objetivo principal deste trabalho é desenvolver um **Assistente Virtual Médico** capaz de auxiliar profissionais de saúde no acesso a informações clínicas contextualizadas. Para isso, o sistema integra cinco pilares fundamentais:

1. **Fine-tuning** do modelo Mistral 7B Instruct v0.3 com o dataset MedQuAD traduzido para português e complementado com QA de doenças brasileiras, utilizando Quantized

Low-Rank Adaptation (QLoRA) 4-bit. O modelo treinado é publicado no Hugging Face Hub[1] para acesso público;

2. **Retrieval-Augmented Generation (RAG)** com 14 Linhas de Cuidado reais e 5 protocolos de emergência sintéticos (IAM, AVC, Sepse, Pneumonia, ITU) indexados no ChromaDB;
3. **Grafo multi-agente** implementado em LangGraph com 7 agentes (6 especializados + logger de auditoria);
4. **Segurança** com guardrails que impedem prescrições diretas e exigem validação humana;
5. **Interface** Gradio para demonstração interativa do assistente.

Aviso de Segurança

Este sistema foi projetado exclusivamente como ferramenta de **suporte à decisão clínica**, nunca como substituto do profissional de saúde. Todas as respostas incluem avisos de validação humana obrigatória e o sistema é impedido, por guardrails, de prescrever medicamentos diretamente.

1.3 Arquitetura Geral do Sistema

A Figura 1.1 apresenta a visão de alto nível da arquitetura, organizada em cinco camadas que correspondem aos cinco pilares do projeto.

[1] https://huggingface.co/zagari/medical-assistant-mistral-7b-ft

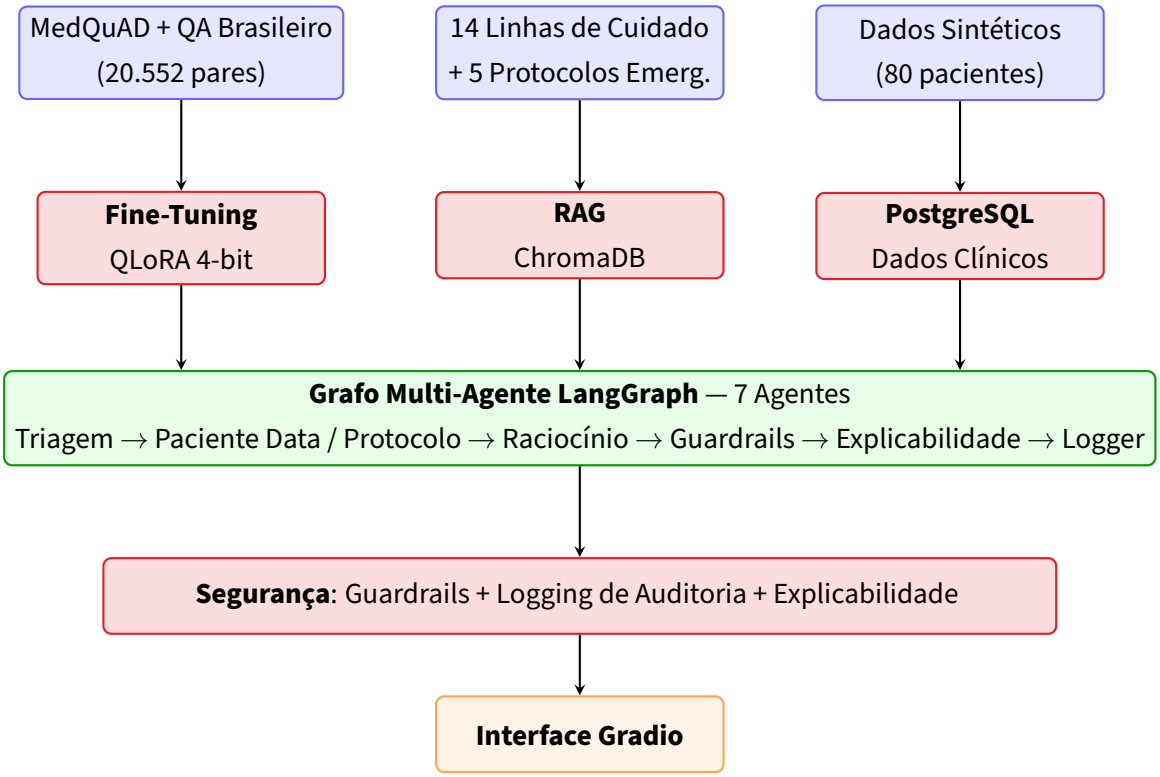

Figura 1.1: Arquitetura geral do Assistente Virtual Médico.

1.4 Quickstart: Testando em 5 Minutos

Para quem deseja apenas **testar o assistente** sem re-treinar o modelo ou regenerar dados, o projeto inclui um script de inicialização rápida. Todos os dados sintéticos e o dataset de treino já estão no repositório, e o modelo fine-tuned é baixado automaticamente do Hugging Face Hub.

1.4.1 Pré-requisitos

- Python 3.11+
- Docker Desktop instalado e rodando
- **GPU NVIDIA (CUDA)** para usar o modelo fine-tuned via LoRA — ou **qualquer máquina** (Mac, Windows, Linux) com o modelo no formato GGUF (ver Seção 3.5.4)

1.4.2 Execução

```
git clone
    ↪ https://github.com/Zagari/virtual-medical-assistant.git
cd virtual-medical-assistant
pip install -r requirements.txt
```

```
bash scripts/quickstart.sh
```

Listing 1.1: Quickstart: do clone ao assistente funcionando.

O script `quickstart.sh` executa automaticamente 5 passos:

1. **Cria o `.env`** a partir do `.env.example` (se não existir), já configurado para baixar o modelo do Hugging Face Hub;
2. **Detecta o hardware:** se há GPU NVIDIA (CUDA), usa o modelo LoRA via `transformers`; caso contrário, instala automaticamente o `llama-cpp-python` (com aceleração Metal no Apple Silicon) para carregar o modelo GGUF;
3. **Sobe o PostgreSQL** via Docker e popula com os 80 pacientes sintéticos (exames, prontuários, receitas);
4. **Indexa os protocolos clínicos** no ChromaDB (14 Linhas de Cuidado + 5 protocolos de emergência);
5. **Inicia a interface Gradio** em `http://localhost:7860`.

O que NÃO precisa ser re-executado

Os seguintes artefatos já estão commitados no repositório e não precisam ser regenerados para testar o sistema:

- **Dados sintéticos** (`data/synthetic/`) — 80 pacientes com exames, prontuários e receitas
- **Protocolos em Markdown** (`data/linhas_de_cuidado/extracted/`) — 14 Linhas de Cuidado extraídas dos PDFs
- **Dataset de treino** (`data/processed/training_data.jsonl`) — ∼20.500 pares QA
- **Modelo fine-tuned** — publicado no Hugging Face Hub, download automático na primeira consulta
- **Modelo GGUF** (opcional) — versão quantizada (∼4 GB) para execução sem GPU, baixada automaticamente do Hub quando necessário. A dependência `llama-cpp-python` é instalada automaticamente pelo `quickstart.sh`

Execução sem GPU

O assistente detecta automaticamente o hardware disponível. Em máquinas sem GPU NVIDIA, o sistema tenta carregar a versão GGUF do modelo (se disponível) ou opera em modo "Sem LLM" com respostas baseadas apenas nos protocolos clínicos. Um seletor na interface permite alternar entre os modelos disponíveis.

1.5 Guia de Reprodução Passo a Passo

O projeto foi concebido para ser **integralmente reprodutível**. Um pipeline de 7 scripts numerados, localizado no diretório `scripts/`, permite que se reconstrua todo o sistema do zero, desde a geração de dados sintéticos até a execução do assistente completo.

1.5.1 Pipeline de Scripts

A Tabela 1.1 descreve cada script, suas pré-condições e saídas.

Tabela 1.1: Pipeline de reprodução: scripts numerados.

#	Script	O que faz	Saída
01	`gerar_dados_sinteticos`	Gera pacientes, exames, prontuários, receitas e 5 protocolos de emergência	`synthetic/`
02	`setup_postgres`	Cria container Docker PostgreSQL e popula com dados sintéticos	Banco operacional
03	`preparar_dataset`	Baixa MedQuAD, traduz para PT-BR, adiciona QA de doenças brasileiras e formata em Alpaca	`training_data.jsonl`
–	*Pausa: Fine-tuning*	*QLoRA via notebook (Colab ou local)*	`models/`
04	`indexar_protocolos`	Chunking dos protocolos Markdown e indexação no ChromaDB	`chroma_db/`
05	`avaliar_baseline`	Executa perguntas médicas no Mistral 7B base (suporta `--tag` para múltiplos conjuntos)	`baseline*.json`
05b	`extrair_amostra_validacao`	Extrai 50 amostras do split de validação com respostas de referência	`validacao_amostra.json`
06	`avaliar_finetuned`	Avalia modelos fine-tuned (multi-checkpoint) com BLEU, ROUGE e similaridade semântica	Relatórios comparativos
06b	`comparar_modelos`	Comparação unificada: baseline vs epoch 2 vs epoch 5	`comparacao_completa.*`
07	`iniciar_app`	Inicia o assistente completo com interface Gradio	Servidor web local
08	`exportar_gguf`	Exporta modelo(s) para GGUF (suporta `--model-path` e `--gguf-filename`)	`models/gguf/`

1.5.2 Fluxo de Execução

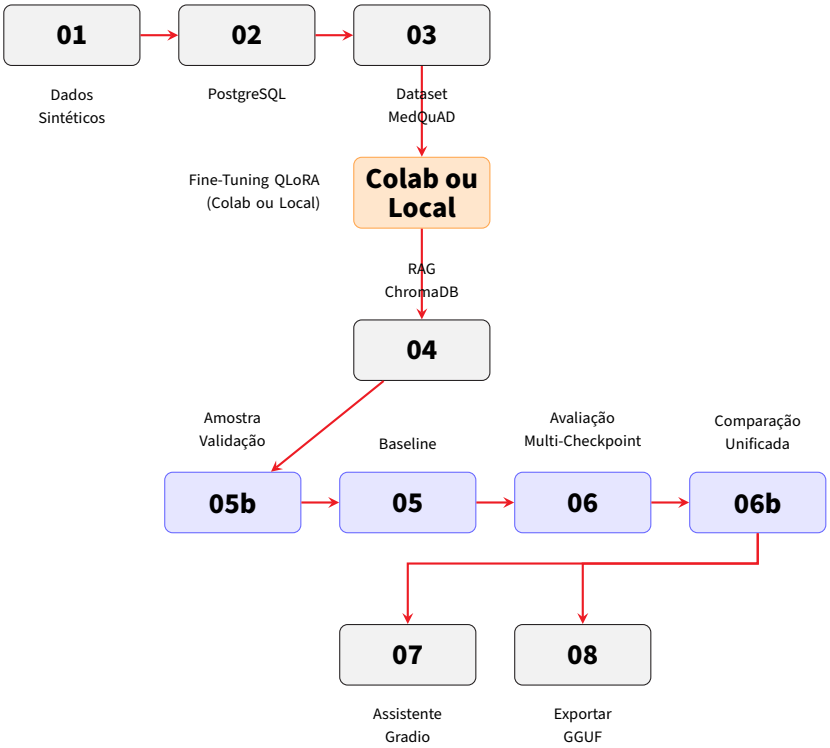

Figura 1.2: Fluxo de execução do pipeline de reprodução.

1.5.3 Como Reproduzir

```
# 0. Pre-requisitos: Python 3.11+, Docker Desktop
git clone
   ↪ https://github.com/Zagari/virtual-medical-assistant.git
cd virtual-medical-assistant
pip install -r requirements.txt
cp .env.example .env    # Configurar variaveis de ambiente

# 1. Gerar dados sinteticos (80 pacientes ficticios)
python scripts/01_gerar_dados_sinteticos.py

# 2. Subir PostgreSQL e popular banco
python scripts/02_setup_postgres.py

# 3. Preparar dataset (MedQuAD + QA doencas brasileiras)
python scripts/03_preparar_dataset.py

# --- PAUSA: Fine-tuning ---
# Opcao A: Usar modelo pre-treinado do Hugging Face Hub
```

```
#   O .env.example ja aponta para o modelo publicado
#   (download automatico na primeira execucao)
# Opcao B: Google Colab (GPU gratuita)
#   Abrir notebooks/03_fine_tuning.ipynb no Colab
#   Upload de data/processed/training_data.jsonl
# Opcao C: Servidor local (GPU 16GB+)
#   Executar notebooks/03_fine_tuning_local.ipynb

# 4. Indexar protocolos clinicos no ChromaDB
python scripts/04_indexar_protocolos.py

# 5-6. Avaliacao multi-checkpoint (baseline vs epoch 2 vs epoch
   ↪ 5)
# Trilha A: 50 perguntas manuais (sobreposicao entre modelos)
python scripts/05_avaliar_baseline.py
python scripts/06_avaliar_finetuned.py \
  --model-path models/checkpoints/checkpoint-2000 --tag epoch2
python scripts/06_avaliar_finetuned.py \
  --model-path models/medical-assistant-final --tag epoch5

# Trilha B: 50 amostras validacao MedQuAD (vs ground truth)
python scripts/05b_extrair_amostra_validacao.py
python scripts/05_avaliar_baseline.py \
  --questions-file data/evaluation/validacao_amostra.json --tag
     ↪ val
python scripts/06_avaliar_finetuned.py \
  --model-path models/checkpoints/checkpoint-2000 --tag
     ↪ epoch2_val \
  --questions-file data/evaluation/validacao_amostra.json \
  --baseline-file data/evaluation/baseline_responses_val.json
python scripts/06_avaliar_finetuned.py \
  --model-path models/medical-assistant-final --tag epoch5_val \
  --questions-file data/evaluation/validacao_amostra.json \
  --baseline-file data/evaluation/baseline_responses_val.json

# Comparacao unificada dos 3 modelos
python scripts/06b_comparar_modelos.py

# 7. Iniciar o assistente
python scripts/07_iniciar_app.py
```

```
# 8. Exportar modelos GGUF (execucao sem GPU)
python scripts/08_exportar_gguf.py \
  --model-path models/checkpoints/checkpoint-2000 \
  --gguf-filename medical-assistant-epoch2-Q4_K_M.gguf
python scripts/08_exportar_gguf.py \
  --model-path models/medical-assistant-final \
  --gguf-filename medical-assistant-epoch5-Q4_K_M.gguf
```

Listing 1.2: Comandos para reprodução completa do projeto.

Princípios do Pipeline

Cada script é **autocontido** e pode ser executado independentemente. Todos imprimem progresso durante a execução (registros gerados, tempo decorrido) e **validam pré-condições** antes de iniciar (por exemplo, o script 02 verifica se os arquivos de dados sintéticos existem antes de tentar popular o banco).

1.6 Estrutura de Arquivos do Projeto

```
virtual-medical-assistant/
|-- README.md
|-- requirements.txt
|-- .env.example
|-- scripts/                    # Pipeline numerado (01 a 07)
|-- data/
|   |-- raw/                    # MedQuAD original
|   |-- processed/              # Dataset hibrido (PT + EN + BR)
|   |-- evaluation/             # Avaliacoes multi-checkpoint +
  ↪ ground truth
|   |-- linhas_de_cuidado/      # 14 PDFs + extracted/
  ↪ (Markdown)
|   +-- synthetic/              # Pacientes, exames, protocolos
  ↪ emerg.
|-- notebooks/
|   |-- 03_fine_tuning.ipynb        # Notebook para Google Colab
|   +-- 03_fine_tuning_local.ipynb # Notebook para execucao
  ↪ local
|-- src/
|   |-- data/                   # Geracao e preprocessamento
|   |-- fine_tuning/            # Configuracao e treino
|   |-- assistant/              # LLM wrapper, RAG, prompts
```

```
|   |-- flows/                    # LangGraph (state, graph,
    ↪ agents/)
|   |-- security/                 # Guardrails, logging, auditoria
|   +-- database/                 # SQLAlchemy models e queries
|-- app/                          # Interface Gradio
|-- docs/                         # Relatorio LaTeX
+-- tests/                        # Testes automatizados
```

Listing 1.3: Estrutura de diretórios do projeto.

1.7 Organização do Documento

O restante deste relatório está organizado da seguinte forma:

- **Capítulo 2** apresenta os dados utilizados: as 14 Linhas de Cuidado, a geração de dados sintéticos, o banco de dados PostgreSQL e o dataset MedQuAD;
- **Capítulo 3** detalha o processo de fine-tuning do Mistral 7B com QLoRA;
- **Capítulo 4** descreve a implementação do RAG com ChromaDB;
- **Capítulo 5** apresenta o grafo multi-agente com LangGraph;
- **Capítulo 6** aborda a segurança, guardrails e explicabilidade;
- **Capítulo 7** traz a avaliação comparativa multi-checkpoint (baseline vs epoch 2 vs epoch 5) com duas trilhas: sobreposição entre modelos e qualidade contra *ground truth* com similaridade semântica;
- **Capítulo 8** conclui com reflexões, limitações e trabalhos futuros.

2

Dados e Preparação

Este capítulo descreve as quatro fontes de dados do projeto: os protocolos clínicos reais (Linhas de Cuidado), os dados sintéticos de pacientes, o banco de dados PostgreSQL e o dataset MedQuAD para fine-tuning.

2.1 Linhas de Cuidado (Protocolos Clínicos)

A base de conhecimento clínico do sistema é composta por **14 Linhas de Cuidado** — protocolos clínicos profissionais em português brasileiro, cobrindo condições que vão de hipertensão e diabetes até demência e polifarmácia. Estes protocolos são documentos reais de prática clínica, com estrutura padronizada que inclui seções como Considerações Iniciais, Diagnóstico, Monitoramento, Encaminhamento e Indicadores.

2.1.1 Protocolos Disponíveis

A Tabela 2.1 lista os 14 protocolos utilizados como base de conhecimento para o sistema RAG.

Tabela 2.1: 14 Linhas de Cuidado indexadas no sistema.

#	Pathway	Domínio	Exemplos de Conteúdo
1	Hipertensão	Cardiovascular	Classificação PA, MAPA, anti-hipertensivos
2	Diabetes	Endócrino	DM1/DM2, HbA1C, insulinização
3	Asma	Respiratório	Classificação gravidade, step therapy
4	Depressão	Psiquiátrico	PHQ-9, ISRS, psicoterapia
5	Ansiedade	Psiquiátrico	GAD-7, benzodiazepínicos, TCC
6	DPOC	Respiratório	GOLD, espirometria, oxigenoterapia
7	Demência	Neurológico	Mini-Mental, anticolinesterásicos
8	Dispepsia	Gastroenterologia	H. pylori, IBP, endoscopia
9	Hipotireoidismo	Endócrino	TSH, levotiroxina, seguimento
10	Insuf. Cardíaca	Cardiovascular	NYHA, IECA, betabloqueadores
11	Obesidade	Metabólico	IMC, dietoterapia, cirurgia bariátrica
12	Pac. Cirúrgico	Cirúrgico	Pré-operatório, risco cirúrgico
13	Polifarmácia	Geriatria	Reconciliação medicamentosa, Beers
14	Tabagismo	Preventivo	Fagerström, TRN, bupropiona

2.1.2 Extração e Limpeza dos Protocolos

Os 14 protocolos foram disponibilizados originalmente em formato PDF. Para viabilizar a indexação no sistema RAG, foi realizado um processo de extração e limpeza:

1. **Extração de texto:** Uso da biblioteca PyMuPDF (fitz) para extrair o conteúdo textual de cada página dos PDFs;
2. **Identificação de seções:** Detecção automática de seções principais por meio de heurística baseada em texto em caixa alta (*ALL CAPS*), seguindo o padrão estrutural dos documentos;
3. **Limpeza:** Remoção de referências ao sistema de origem e caracteres indesejados;
4. **Saída em Markdown:** Geração de arquivos `.md` com hierarquia de seções preservada (## para seções, ### para subseções), facilitando o chunking posterior.

O resultado são 14 arquivos Markdown limpos, totalizando aproximadamente 448 KB de conteúdo clínico estruturado, armazenados em `data/linhas_de_cuidado/extracted/`. Estes arquivos são versionados no repositório e servem como entrada direta para o pipeline de RAG (Capítulo 4).

Nota sobre a Extração

O script de extração de PDFs é um utilitário auxiliar executado uma única vez fora do pipeline principal. Os arquivos Markdown resultantes já estão incluídos no repositório, portanto **não é necessário reexecutar** a extração — basta utilizar os arquivos `.md` já disponíveis.

2.1.3 Protocolos Complementares de Emergência

Para cobrir cenários de **emergência** não presentes nas 14 Linhas de Cuidado (que focam em condições crônicas), o sistema inclui 5 protocolos sintéticos adicionais. Estes protocolos foram elaborados com base em diretrizes reconhecidas (AHA, Surviving Sepsis Campaign, SBPT, IDSA) e seguem a mesma estrutura dos protocolos extraídos.

Tabela 2.2: Protocolos de emergência sintéticos complementares.

#	Protocolo	CID	Conteúdo Principal
1	IAM (Infarto Agudo do Miocárdio)	I21	MONABCH, reperfusão, tempo porta-balão
2	AVC (Acidente Vascular Cerebral)	I64	NIHSS, trombólise rtPA, janela terapêutica
3	Sepse e Choque Séptico	A41	Hour-1 Bundle, qSOFA, ressuscitação
4	Pneumonia (PAC)	J18	CURB-65, antibioticoterapia empírica
5	ITU (Infecção do Trato Urinário)	N39	Cistite vs pielonefrite, bacteriúria

Cada protocolo sintético inclui as seções: Considerações Iniciais, Diagnóstico, Manejo Inicial, Monitoramento e Indicadores de Qualidade. Os arquivos são gerados automaticamente pelo script `01_gerar_dados_sinteticos` e salvos em formato Markdown no diretório `data/synthetic/protocolos_complementares/`, prontos para indexação no ChromaDB junto com as Linhas de Cuidado.

Protocolos Sintéticos

Os protocolos de emergência são **sintéticos**, criados para fins educacionais e de demonstração. Em ambiente de produção, devem ser substituídos por protocolos institucionais validados pela equipe médica.

2.2 Dados Sintéticos de Pacientes

Para simular um ambiente hospitalar realista, o sistema utiliza dados de pacientes inteiramente fictícios, gerados com coerência clínica. A geração é realizada pelo primeiro script

do pipeline (`01_gerar_dados_sinteticos`), que invoca o módulo `synthetic_generator` localizado em `src/data/`.

2.2.1 Geração de Pacientes

Foram gerados **80 pacientes fictícios** utilizando a biblioteca Faker (localização `pt_BR`), com sementes fixas (`seed=42`) para garantir a reprodutibilidade. Os dados pessoais incluem:

Tabela 2.3: Campos do cadastro de pacientes sintéticos.

Campo	Exemplo	Geração
nome	"Maria Santos de Oliveira"	`Faker.name()`
cpf	"123.456.789-00"	`Faker.cpf()`
data_nascimento	1958-03-15	`Faker.date_of_birth()`
sexo	"F"	Distribuição 55%F / 45%M
telefone	"(11) 98765-4321"	`Faker.phone_number()`
endereço	"Rua das Flores, 123"	`Faker.address()`
comorbidades	["hipertensão", "DM2"]	Distribuição probabilística
alergias	["dipirona"]	Lista aleatória
medicamentos_uso	["losartana 50mg"]	Coerente com comorbidades

2.2.2 Coerência Clínica

Um diferencial importante da geração de dados é a **coerência clínica** entre comorbidades, medicamentos, exames e consultas. O gerador implementa uma base de conhecimento médico com 9 condições alinhadas às Linhas de Cuidado:

Tabela 2.4: Distribuição de condições nos dados sintéticos.

Condição	Prevalência	Exemplos de Medicamentos Associados
Hipertensão	45%	Losartana 50mg, enalapril 10mg, anlodipino 5mg
Diabetes tipo 2	30%	Metformina 850mg, glibenclamida 5mg, insulina NPH
Obesidade	25%	Sibutramina 10mg, orlistate 120mg
Depressão	20%	Sertralina 50mg, fluoxetina 20mg, escitalopram 10mg
Ansiedade	15%	Escitalopram 10mg, alprazolam 0,5mg
Asma	12%	Salbutamol spray, budesonida 200mcg
Hipotireoidismo	10%	Levotiroxina 50mcg, levotiroxina 75mcg
DPOC	8%	Tiotrópio 18mcg, formoterol + budesonida
Insuf. Cardíaca	5%	Enalapril 10mg, furosemida 40mg, carvedilol 12,5mg

Muitos pacientes possuem **múltiplas comorbidades** (por exemplo, hipertensão + diabetes + obesidade), o que é clinicamente realista e cria cenários ricos para o assistente consultar múltiplos protocolos simultaneamente.

2.2.3 Dados Clínicos Gerados

Para cada paciente, o gerador cria dados clínicos coerentes com suas comorbidades:

Tabela 2.5: Volume de dados clínicos gerados.

Tipo	Total Gerado	Arquivo de Saída
Pacientes	80	`pacientes.json`
Exames	199	`exames.json`
Prontuários	297	`consultas.json`
Receitas	163	`receitas.json`

Exemplos de coerência clínica implementada:

- Paciente **diabético** → exames de HbA1C (valores entre 5,5–12%), glicemia de jejum (80–250 mg/dL), creatinina;
- Paciente **hipertenso** → MAPA 24h com valores sistólico/diastólico realistas, ecocardiograma;
- Paciente **asmático** → espirometria com VEF1/CVF e classificação de gravidade;
- Paciente com **depressão** → avaliação PHQ-9 com escore de gravidade e notas clínicas compatíveis.

2.3 Banco de Dados PostgreSQL

Os dados sintéticos são armazenados em um banco PostgreSQL 16, executado em container Docker. O setup é automatizado pelo `scripts/02_setup_postgres.py`.

2.3.1 Schema do Banco

O banco possui 4 tabelas, modeladas com SQLAlchemy ORM:

```sql
-- Pacientes
CREATE TABLE pacientes (
    id UUID PRIMARY KEY,
    nome VARCHAR(200) NOT NULL,
    cpf VARCHAR(14) UNIQUE NOT NULL,
```

```sql
    data_nascimento DATE NOT NULL,
    sexo VARCHAR(1) NOT NULL,
    telefone VARCHAR(20),
    endereco TEXT,
    comorbidades TEXT[],
    alergias TEXT[],
    medicamentos_uso TEXT[]
);

-- Exames clinicos
CREATE TABLE exames (
    id UUID PRIMARY KEY,
    paciente_id UUID REFERENCES pacientes(id),
    tipo VARCHAR(100) NOT NULL,
    data DATE NOT NULL,
    resultados JSONB,        -- Resultados estruturados
    interpretacao TEXT,
    status VARCHAR(20)       -- pendente, realizado, cancelado
);

-- Prontuarios (consultas medicas)
CREATE TABLE prontuarios (
    id UUID PRIMARY KEY,
    paciente_id UUID REFERENCES pacientes(id),
    data DATE NOT NULL,
    diagnostico_cid VARCHAR(10),  -- Codigo CID-10
    diagnostico_texto TEXT,
    observacoes TEXT
);

-- Receitas medicas
CREATE TABLE receitas (
    id UUID PRIMARY KEY,
    paciente_id UUID REFERENCES pacientes(id),
    data DATE NOT NULL,
    condicao TEXT,
    medicamentos TEXT[],
    validade_dias INTEGER,
    observacoes TEXT
);
```

Listing 2.1: Schema do banco de dados.

2.3.2 Características Técnicas

- **UUIDs como chaves primárias:** Evita colisões e permite geração distribuída;
- **JSONB para resultados de exames:** Permite armazenar resultados com estrutura variável (diferentes tipos de exames geram diferentes campos), mantendo a capacidade de consulta;
- **ARRAY para listas:** Comorbidades, alergias e medicamentos são armazenados como arrays nativos do PostgreSQL, facilitando consultas com operador `ANY`;
- **CID-10:** Diagnósticos são codificados no padrão Classificação Internacional de Doenças (CID), permitindo consultas padronizadas;
- **Consultas parametrizadas:** O módulo `src/database/queries.py` utiliza `SQLAlchemy text()` com parâmetros nomeados para prevenir injeção SQL.

2.3.3 Infraestrutura Docker

O script `02_setup_postgres.py` automatiza todo o processo:

1. Verifica se o Docker Desktop está em execução;
2. Cria ou inicia o container `medical_assistant_db` (PostgreSQL 16 Alpine);
3. Cria as 4 tabelas via `SQLAlchemy Base.metadata.create_all()`;
4. Carrega os 4 arquivos JSON gerados pelo script 01;
5. Popula o banco de dados com os dados sintéticos;
6. Executa validação final (contagem de registros por tabela).

2.4 Dataset MedQuAD

O **MedQuAD** (*Medical Question Answering Dataset*) é o dataset base utilizado para o fine-tuning do modelo Mistral 7B. Criado pelo National Institutes of Health (NIH) dos Estados Unidos, o MedQuAD reúne pares de perguntas e respostas médicas extraídas de fontes confiáveis do governo americano.

2.4.1 Escolha da Versão e Limitações de Copyright

Existem múltiplas versões do MedQuAD disponíveis no HuggingFace. Optou-se pela versão `lavita/MedQuAD`, que possui os metadados mais completos. No entanto, uma **limitação importante** foi descoberta durante a implementação:

Remoção de Respostas por Copyright

O dataset `lavita/MedQuAD` contém 47.441 registros, porém **aproximadamente 65% (31.034 registros) possuem o campo `answer=None`**. Isso ocorreu porque as instituições originais (NIH, NCI, etc.) solicitaram a remoção das respostas por questões de copyright, mantendo apenas as perguntas. O dataset utilizável contém **aproximadamente 16.400 pares QA completos**.

Tabela 2.6: Comparação de versões do MedQuAD disponíveis no HuggingFace.

Dataset	Registros	Com Resposta	Observações
`lavita/MedQuAD`	47.441	∼**16.400**	Metadados ricos, 65% sem resposta
`keivalya/MedQuad-...`	16.400	16.400	Versão já filtrada
`Tonic/medquad`	15.549	15.549	Pré-formatado Alpaca

2.4.2 Características do Dataset

Tabela 2.7: Resumo do dataset MedQuAD (versão lavita, registros válidos).

Característica	Descrição
Tamanho original	47.441 registros (65% com `answer=None`)
Tamanho útil	∼**16.400** pares pergunta-resposta completos
Idioma original	Inglês
Fontes	8 institutos NIH: NCI, GHR, MedlinePlus, NIDDK, NINDS, NIHSeniorHealth, NHLBI, CDC
Metadados	Códigos UMLS, tipos semânticos, sinônimos, URLs das fontes
Domínios	Doenças, tratamentos, medicamentos, prevenção, diagnósticos
Qualidade	Alta — respostas baseadas em fontes governamentais revisadas
Licença	Creative Commons BY 4.0

2.4.3 Abordagem Híbrida de Treinamento

Com base em pesquisa sobre fine-tuning multilíngue, adotamos uma **abordagem híbrida** para composição do dataset final, combinando três fontes:

1. **MedQuAD traduzido (80%):** Pares QA traduzidos para português via Google Translate;
2. **MedQuAD original em inglês (20%):** Amostra dos dados originais para preservar a qualidade médica e a capacidade multilíngue do modelo;
3. **QA de medicina brasileira:** Perguntas e respostas nativas em português sobre temas específicos do Brasil.

Fundamentação da Abordagem Híbrida

Pesquisas recentes (2024-2025) demonstram que fine-tuning exclusivamente com dados traduzidos pode introduzir ruído e perder nuances do conhecimento médico original. A inclusão de 20% de dados em inglês, combinada com dados nativos em português, mitiga esses problemas e preserva a capacidade multilíngue do Mistral 7B.

2.4.4 QA de Medicina Brasileira

Para cobrir conhecimento médico específico do Brasil — ausente no MedQuAD americano —, desenvolvemos um conjunto de **55 pares QA** sobre temas brasileiros:

Tabela 2.8: Categorias de QA de medicina brasileira.

Categoria	QA	Exemplos de Tópicos
Arboviroses	6	Dengue, Zika, Chikungunya, diagnóstico diferencial
Malária	5	Regiões endêmicas, P. vivax vs P. falciparum, tratamento
Doença de Chagas	4	Transmissão oral (açaí), benznidazol, cardiopatia
Leishmaniose	3	Tegumentar vs visceral, antimoniato de meglumina
Febre Amarela	2	Vacinação, áreas de recomendação
Acidentes com animais	7	Jararaca, cascavel, aranha-marrom, escorpião
SUS e navegação	5	UBS, CAPS, referência/contrarreferência, alto custo
Medicamentos Brasil	4	Dipirona, RENAME, Farmácia Popular
Vacinação PNI	3	Calendário infantil, febre amarela, dengue
Pré-natal SUS	3	Consultas, exames obrigatórios, diabetes gestacional
Saúde Mental	2	RAPS, tipos de CAPS, manejo de crise
Total	**55**	

Estes QA foram elaborados com base em fontes oficiais: Ministério da Saúde, Protocolos Clínicos e Diretrizes Terapêuticas (PCDT), RENAME 2024 e diretrizes do PNI.

2.4.5 Pipeline de Preparação

O `scripts/03_preparar_dataset.py` executa o seguinte pipeline:

1. **Download:** Baixa o dataset `lavita/MedQuAD` do HuggingFace;
2. **Filtragem:** Remove registros com `answer=None` (∼65% do total);
3. **Tradução:** Traduz perguntas e respostas para português (com checkpoints a cada 1.000 registros);
4. **Amostragem EN:** Seleciona 20% do MedQuAD original em inglês;
5. **QA Brasileiro:** Carrega os 55 pares de medicina brasileira;
6. **Composição:** Combina as três fontes e embaralha;
7. **Formatação Alpaca:** Salva no formato JSONL.

O formato Alpaca utilizado segue a estrutura padrão para instruction tuning:

```
{
    "instruction": "Responda a seguinte pergunta medica de forma
                    clara e detalhada.",
    "input": "Quais sao os sintomas do diabetes tipo 2?",
    "output": "Os sintomas comuns do diabetes tipo 2 incluem..."
}
```

Listing 2.2: Formato Alpaca para fine-tuning.

2.4.6 Composição Final do Dataset

Tabela 2.9: Composição do dataset final de treinamento.

Fonte	Registros	Proporção	Arquivo
MedQuAD traduzido (PT)	∼16.400	78%	`medquad_pt.jsonl`
MedQuAD original (EN)	∼4.100	20%	`medquad_en_sample.jsonl`
QA Brasileiro	55	2%	`brazilian_medical_qa.jsonl`
Total	**∼20.500**	100%	`training_data.jsonl`

Arquivos Versionados

Os arquivos de dataset processado (`medquad_pt.jsonl`, `medquad_en_sample.jsonl`, `brazilian_medical_qa.jsonl` e `training_data.jsonl`) são versionados no repositório Git. Isso permite reproduzir o treinamento sem necessidade de re-executar a tradução, que leva várias horas.

3

Fine-Tuning do Modelo

Este capítulo descreve o processo de fine-tuning do modelo de linguagem para o domínio médico. O objetivo é adaptar um modelo generalista (Mistral 7B) para responder perguntas médicas em português com maior precisão e relevância clínica.

3.1 Modelo Base: Mistral 7B Instruct

3.1.1 Justificativa da Escolha

Para o fine-tuning, escolhemos o **Mistral 7B Instruct v0.3** como modelo base. Esta decisão foi tomada após análise comparativa de alternativas, considerando os seguintes critérios:

Tabela 3.1: Comparação de modelos candidatos para fine-tuning

Modelo	Parâmetros	Contexto	Licença	VRAM (4-bit)
Mistral 7B Instruct	7.3B	32K tokens	Apache 2.0	5GB
LLaMA 2 7B	7B	4K tokens	Meta License	5GB
BioMistral 7B	7.3B	32K tokens	Apache 2.0	5GB
Falcon 7B	7B	2K tokens	Apache 2.0	5GB

Por que não usar BioMistral?

O BioMistral já possui conhecimento médico pré-treinado, o que dificultaria demonstrar o **delta** (melhoria) obtido com o fine-tuning. Ao usar o Mistral 7B generalista, conseguimos evidenciar claramente a diferença entre as respostas antes e depois do treinamento especializado.

3.1.2 Características do Mistral 7B

O Mistral 7B apresenta características técnicas que o tornam ideal para este projeto:

- **Arquitetura**: Transformer decoder-only com 32 camadas
- **Sliding Window Attention**: Janela de 4096 tokens com contexto efetivo de 32K
- **Grouped-Query Attention (GQA)**: Reduz uso de memória mantendo qualidade
- **Instruct-tuned**: Versão já otimizada para seguir instruções
- **Multilíngue**: Bom desempenho em português, mesmo sem treinamento específico

3.2 Dataset: MedQuAD e Abordagem Híbrida

3.2.1 Escolha do Dataset

Para o fine-tuning, utilizamos o **MedQuAD** (Medical Question Answering Dataset), um dataset de perguntas e respostas médicas criado pelo National Institutes of Health (NIH) dos Estados Unidos, combinado com uma abordagem híbrida que inclui dados em inglês e conhecimento médico brasileiro.

Tabela 3.2: Comparação de versões do MedQuAD disponíveis

Dataset (HuggingFace)	Registros	Com Resposta	Escolhido
`lavita/MedQuAD`	47.441	∼16.400	✓
`keivalya/MedQuad-...`	16.400	16.400	
`Tonic/medquad`	15.549	15.549	

Limitação do Dataset

O dataset `lavita/MedQuAD` possui 47.441 registros, porém ∼**65% (31.034 registros) têm `answer=None`** devido a remoções por copyright solicitadas pelas instituições originais. O dataset utilizável contém aproximadamente **16.400 pares QA completos**.

Optamos pela versão `lavita/MedQuAD` por oferecer:

- **Metadados ricos**: códigos UMLS, tipos semânticos, sinônimos
- **Fontes documentadas**: NCI, GHR, MedlinePlus, NIDDK, NINDS, NIHSeniorHealth, NHLBI, CDC
- **Licença clara**: Creative Commons BY 4.0

3.2.2 Fontes do MedQuAD

O dataset original do MedQuAD compila informações de 8 fontes do NIH, cobrindo diversas especialidades médicas:

Tabela 3.3: Fontes do dataset MedQuAD e cobertura temática

Fonte	**Cobertura**	**Pares QA (úteis)**
National Cancer Institute (NCI)	Oncologia	∼250
Genetics Home Reference (GHR)	Genética	∼5.400
MedlinePlus Health Topics	Saúde geral	∼980
NIDDK	Diabetes, digestivo, renal	∼1.100
NINDS	Neurologia	∼1.100
NIHSeniorHealth	Geriatria	∼750
NHLBI	Cardiologia, pneumologia	∼550
CDC	Doenças infecciosas	∼270
Total (com resposta)		**∼16.400**

3.2.3 Abordagem Híbrida: PT + EN + Brasileiro

Com base em pesquisa sobre fine-tuning multilíngue, adotamos uma abordagem híbrida que combina três fontes de dados para maximizar a qualidade do modelo:

1. **MedQuAD traduzido (78%):** ∼16.400 pares traduzidos para português
2. **MedQuAD original em inglês (20%):** ∼4.100 pares preservados no idioma original
3. **QA de medicina brasileira (2%):** 55 pares sobre doenças endêmicas, SUS, medicamentos brasileiros

Fundamentação da Abordagem Híbrida

Estudos recentes demonstram que: (1) dados traduzidos automaticamente introduzem ruído que o modelo aprende a imitar; (2) fine-tuning exclusivo em um idioma pode degradar capacidades multilíngues; (3) apenas 40 exemplos multilíngues preservam capacidades cross-lingual. Nossa proporção de 20% em inglês mitiga esses problemas.

3.2.4 Tradução para Português

O dataset MedQuAD original está em inglês. Para treinar o modelo a responder em português brasileiro, implementamos um pipeline de tradução automatizada.

Estratégia de Tradução

Utilizamos a biblioteca `deep-translator` que oferece acesso gratuito à API do Google Translate:

```python
from deep_translator import GoogleTranslator

def translate_to_portuguese(text: str) -> str:
    """Traduz texto de ingles para portugues brasileiro."""
    if not text or len(text.strip()) == 0:
        return text

    translator = GoogleTranslator(source='en', target='pt')

    # Textos longos precisam ser divididos (limite de 5000 chars)
    if len(text) > 4500:
        chunks = [text[i:i+4500] for i in range(0, len(text), 4500)]
        translated_chunks = [translator.translate(chunk) for chunk in chunks]
        return ' '.join(translated_chunks)

    return translator.translate(text)
```

Listing 3.1: Função de tradução com tratamento de textos longos

A tradução dos ∼16.400 registros leva aproximadamente **4–6 horas** devido ao volume de chamadas à API do Google Translate e ao *rate limiting* imposto pelo serviço. O sistema de checkpoints descrito a seguir permite retomar o processo em caso de interrupção.

Dataset já traduzido disponível no repositório

Para evitar que cada usuário precise repetir o processo de tradução, o dataset final já traduzido e formatado está commitado no repositório em `data/processed/`:

- `medquad_pt.jsonl` — MedQuAD traduzido para português (∼16.400 pares)
- `medquad_en_sample.jsonl` — Amostra em inglês (∼4.100 pares)
- `brazilian_medical_qa.jsonl` — QA de doenças brasileiras (55 pares)
- `training_data.jsonl` — Dataset final híbrido (∼20.500 pares, pronto para treino)

O script `03_preparar_dataset.py` só precisa ser re-executado se o usuário desejar regenerar a tradução ou alterar a composição do dataset.

Sistema de Checkpoints

Dado o volume do dataset (∼16.400 registros úteis), implementamos um sistema de checkpoints que salva o progresso a cada 1.000 traduções. Isso permite:

- Retomar o processo em caso de interrupção
- Monitorar o progresso da tradução
- Evitar retrabalho em caso de erros de rede

```
CHECKPOINT_INTERVAL = 1000

for i, row in enumerate(dataset):
    # Pular registros sem resposta (copyright)
    if row['answer'] is None:
        continue

    # Traduzir pergunta e resposta
    translated_question = translate_to_portuguese(row['question'])
    translated_answer = translate_to_portuguese(row['answer'])

    translated_data.append({
        "instruction": "Responda a seguinte pergunta medica...",
        "input": translated_question,
        "output": translated_answer
```

```
    })

    # Salvar checkpoint a cada 1000 registros
    if (i + 1) % CHECKPOINT_INTERVAL == 0:
        save_checkpoint(translated_data, i + 1)
```

Listing 3.2: Sistema de checkpoint para tradução

3.2.5 Formato Alpaca

O dataset traduzido é formatado no padrão **Alpaca**, amplamente utilizado para instruction tuning de LLMs:

```
{
    "instruction": "Responda a seguinte pergunta medica de forma
                    clara e detalhada.",
    "input": "Quais sao os sintomas do diabetes tipo 2?",
    "output": "Os sintomas mais comuns do diabetes tipo 2 incluem:
                aumento da sede (polidipsia), miccao frequente
                (poliuria), fadiga, visao turva, cicatrizacao lenta
                de feridas, formigamento nas maos e pes..."
}
```

Listing 3.3: Estrutura do formato Alpaca

O template de prompt utilizado durante o treinamento é:

Template Alpaca em Português

```
Abaixo está uma instrução que descreve uma tarefa, junto com uma entrada
que fornece contexto adicional.  Escreva uma resposta que complete
adequadamente a solicitação.

### Instrução:
{instruction}

### Entrada:
{input}

### Resposta:
{output}
```

O arquivo final é salvo no formato JSONL (JSON Lines), com um registro por linha:

```
{"instruction": "Responda...", "input": "O que e hipertensao?",
   ↪ "output": "A hipertensao arterial..."}
{"instruction": "Responda...", "input": "Quais as causas do
   ↪ diabetes?", "output": "O diabetes tipo 1..."}
```

Listing 3.4: Exemplo do arquivo training_data.jsonl

3.3 Técnica: QLoRA

Para realizar o fine-tuning de forma eficiente em hardware limitado (Google Colab Free com GPU T4), utilizamos a técnica **QLoRA** (Quantized Low-Rank Adaptation).

3.3.1 Fundamentos do LoRA

O Low-Rank Adaptation (LoRA) é uma técnica de fine-tuning eficiente que, em vez de atualizar todos os parâmetros do modelo, treina apenas matrizes de baixo rank que são adicionadas às camadas existentes:

$$W' = W + \Delta W = W + BA \tag{3.1}$$

Onde:

- $W \in \mathbb{R}^{d \times k}$ é a matriz de pesos original (congelada)
- $B \in \mathbb{R}^{d \times r}$ e $A \in \mathbb{R}^{r \times k}$ são as matrizes treináveis
- $r \ll \min(d, k)$ é o *rank* do LoRA

3.3.2 Quantização 4-bit

O QLoRA combina LoRA com quantização de 4 bits, reduzindo drasticamente o uso de memória:

Tabela 3.4: Comparação de uso de memória por técnica de fine-tuning

Técnica	VRAM (Mistral 7B)	Parâmetros Treináveis
Full Fine-Tuning (FP16)	28 GB	7.3B (100%)
LoRA (FP16)	14 GB	42M (0.6%)
QLoRA (4-bit)	**5 GB**	**42M (0.6%)**

A configuração de quantização utilizada:

```python
from transformers import BitsAndBytesConfig

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,              # Quantizacao 4-bit
    bnb_4bit_quant_type="nf4",      # Normal Float 4-bit
    bnb_4bit_compute_dtype=torch.float16,  # Computacao em FP16
    bnb_4bit_use_double_quant=True  # Quantizacao dupla (economia extra)
)
```

Listing 3.5: Configuração de quantização 4-bit

3.3.3 Configuração do LoRA

Os hiperparâmetros do LoRA foram escolhidos para maximizar a capacidade de aprendizado dentro das limitações de hardware:

```python
from peft import LoraConfig

lora_config = LoraConfig(
    r=64,                       # Rank (maior = mais capacidade)
    lora_alpha=128,             # Scaling factor (2x o rank)
    lora_dropout=0.05,          # Regularizacao
    bias="none",                # Nao treinar bias
    task_type="CAUSAL_LM",      # Modelagem de linguagem causal

    # Camadas a serem treinadas
    target_modules=[
        "q_proj", "k_proj", "v_proj", "o_proj",  # Attention
        "gate_proj", "up_proj", "down_proj"       # MLP
    ]
)
```

Listing 3.6: Configuração LoRA para o fine-tuning

Escolha do Rank

O rank $r = 64$ foi escolhido como um equilíbrio entre capacidade de aprendizado e uso de memória. Ranks maiores (128, 256) podem capturar mais nuances, mas aumentam significativamente o tempo de treino e o risco de overfitting. Para domínios especializados como medicina, $r \in [32, 64]$ é geralmente suficiente.

3.4 Processo de Treino

3.4.1 Ambiente de Execução

O projeto disponibiliza **dois notebooks** para o fine-tuning, permitindo flexibilidade na escolha do ambiente:

Tabela 3.5: Notebooks disponíveis para fine-tuning

Notebook	Ambiente	Requisitos
`03_fine_tuning.ipynb`	Google Colab Free	Conta Google
`03_fine_tuning_local.ipynb`	Servidor/PC local	GPU 16GB+ VRAM

O notebook para **Google Colab** é otimizado para sessões limitadas (checkpoints frequentes, Google Drive). O notebook **local** auto-detecta a GPU disponível e ajusta batch size automaticamente, suportando também Apple Silicon (MPS).

Para o ambiente Colab Free, as especificações são:

Tabela 3.6: Especificações do ambiente de treinamento

Componente	Especificação
Plataforma	Google Colab Free
GPU	NVIDIA Tesla T4
VRAM	15 GB
RAM	12.7 GB
Limite de sessão	12 horas
Armazenamento	Google Drive (15 GB free)

3.4.2 Hiperparâmetros de Treinamento

Os hiperparâmetros foram configurados para otimizar a qualidade do modelo dentro das restrições de hardware e tempo:

```
from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir="./checkpoints",

    # Epochs e batching
    num_train_epochs=5,                    # 5 epochs para
        ↪ qualidade
```

```
    per_device_train_batch_size=2,          # Batch size (limite T4)
    gradient_accumulation_steps=8,          # Effective batch = 16

    # Learning rate
    learning_rate=2e-4,                     # Taxa padrao para LoRA
    warmup_ratio=0.03,                      # 3% warmup
    weight_decay=0.01,                      # Regularizacao L2
    lr_scheduler_type="cosine",             # Decay suave

    # Checkpoints (CRITICO para Colab Free!)
    save_strategy="steps",
    save_steps=500,                         # Salvar a cada ~30-40 min
    save_total_limit=3,                     # Manter 3 checkpoints

    # Avaliacao
    evaluation_strategy="steps",
    eval_steps=500,
    load_best_model_at_end=True,

    # Precisao
    fp16=True,                              # Mixed precision
)
```

Listing 3.7: Configuração completa do treinamento

Tabela 3.7: Resumo dos hiperparâmetros de treinamento

Parâmetro	**Valor**	**Justificativa**
Epochs	5	Qualidade máxima, convergência completa
Batch size efetivo	16	2×8 (grad accumulation)
Learning rate	2×10^{-4}	Padrão para LoRA
Warmup	3%	Estabilidade inicial
LR scheduler	Cosine	Decay suave até o final
Checkpoints	500 steps	Recuperação de sessão

3.4.3 Sistema de Checkpoints para Colab Free

Uma limitação crítica do Google Colab Free é o limite de sessão de aproximadamente 12 horas. Para um treino de 5 epochs com ∼20.500 registros (dataset híbrido), o tempo total estimado é de 10-15 horas, exigindo múltiplas sessões.

Estratégia de Múltiplas Sessões

O treino completo requer 3-4 sessões do Colab. Checkpoints são salvos a cada 500 steps (30-40 minutos) no Google Drive, permitindo retomar o treino de onde parou.

O fluxo de múltiplas sessões é ilustrado abaixo:

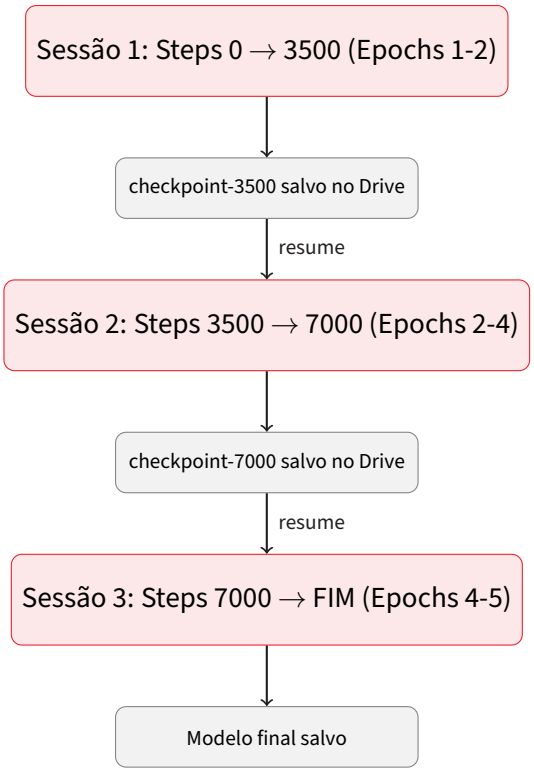

Figura 3.1: Fluxo de treinamento com múltiplas sessões no Colab Free

O código para retomar o treino de um checkpoint:

```
import os

def find_latest_checkpoint(checkpoint_dir):
    """Encontra o checkpoint mais recente."""
    checkpoints = [d for d in os.listdir(checkpoint_dir)
                   if d.startswith("checkpoint-")]
    if not checkpoints:
        return None
```

```
    latest = sorted(checkpoints, key=lambda x: int(x.split("-")[1]))[-1]
    return os.path.join(checkpoint_dir, latest)

# Verificar e retomar
latest_checkpoint = find_latest_checkpoint(CHECKPOINT_DIR)

if latest_checkpoint:
    print(f"Retomando de: {latest_checkpoint}")
    trainer.train(resume_from_checkpoint=latest_checkpoint)
else:
    print("Iniciando treino do zero...")
    trainer.train()
```

Listing 3.8: Retomando treino de checkpoint existente

3.4.4 Execução Local

Para usuários com acesso a GPUs locais (RTX 3090, 4090, A100, etc.), o notebook `03_fine_tuning_local.` oferece vantagens:

- **Sem limite de sessão**: Treino contínuo sem interrupções
- **Auto-detecção de GPU**: Ajusta batch size conforme VRAM disponível
- **Paths locais**: Usa diretórios do projeto diretamente
- **Checkpoints menos frequentes**: 1000 steps (vs 500 no Colab)

Tabela 3.8: Configuração automática por GPU

GPU (VRAM)	Batch Size	Grad Accum	Batch Efetivo
24GB+ (A100, RTX 4090)	4	4	16
16-24GB (RTX 3090, A10)	2	8	16
<16GB	1	16	16

3.4.5 Resultados do Treinamento

O treinamento foi executado em servidor local com GPU NVIDIA RTX A5000 (24GB VRAM). A Tabela 3.9 apresenta os resultados obtidos.

Tabela 3.9: Resultados do treinamento

Métrica	**Valor**
Steps totais	5.780
Loss final (treino)	6,79
Tempo total	∼55 horas
Throughput	0,47 samples/s
Parâmetros treináveis	167M (4,27% do total)
Tamanho dos adapters	∼640 MB

Tabela 3.10: Tempo de treinamento por configuração

Ambiente	**GPU**	**Epochs**	**Tempo**	**Sessões**
Local	**RTX A5000 (24GB)**	**5**	**∼55h**	**1**
Colab Free	T4 (15GB)	5	10-15h	2
Local	RTX 4090 (24GB)	5	3-4h	1
Local	A100 (40GB)	5	2-3h	1

3.5 Persistência do Modelo

3.5.1 Salvando os Adapters LoRA

Após o treinamento, salvamos apenas os **adapters LoRA** (não o modelo completo). Isso resulta em arquivos muito menores:

Tabela 3.11: Comparação de tamanho de arquivos salvos

Tipo de Salvamento	**Tamanho Aproximado**
Modelo completo (merged)	15-28 GB
Adapters LoRA apenas	**500 MB**

```
# Salvar adapters LoRA
SAVE_PATH = "/content/drive/MyDrive/medical-assistant-final"
model.save_pretrained(SAVE_PATH)
tokenizer.save_pretrained(SAVE_PATH)
```

Listing 3.9: Salvando o modelo fine-tuned

3.5.2 Publicação no Hugging Face Hub

Os adapters LoRA (∼640 MB) excedem o limite de 100 MB por arquivo do GitHub. Para garantir acesso público ao modelo treinado, os adapters são publicados no **Hugging Face Hub**, o repositório padrão da comunidade para modelos de linguagem.

Ambos os notebooks incluem uma célula dedicada ao upload:

```
from huggingface_hub import HfApi, login

login(token=os.getenv("HF_TOKEN"))
api = HfApi()
api.create_repo(repo_id=HF_HUB_REPO, exist_ok=True)
api.upload_folder(
    folder_path=FINAL_MODEL_DIR,
    repo_id=HF_HUB_REPO,
    commit_message="Upload adapters LoRA fine-tuned",
)
```

Listing 3.10: Upload do modelo para o Hugging Face Hub

O modelo publicado inclui um *model card* (README.md) com descrição completa: hiperparâmetros, dados de treino, resultados, instruções de uso e limitações.

3.5.3 Carregando o Modelo para Uso

O modelo pode ser carregado de duas fontes — path local ou diretamente do Hugging Face Hub:

```
from transformers import AutoModelForCausalLM, AutoTokenizer

# Opcao 1: path local
model_id = "models/medical-assistant-final"
# Opcao 2: Hugging Face Hub (download automatico)
model_id = "zagari/medical-assistant-mistral-7b-ft"

tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    torch_dtype="float16",
    device_map="auto",
)
```

Listing 3.11: Carregando o modelo fine-tuned para inferência

Na aplicação, a variável de ambiente `FINE_TUNED_MODEL` controla a origem. O `.env.example` do projeto já vem configurado com o repositório do Hub, permitindo que quem clonar o projeto utilize o modelo sem precisar re-treinar.

3.5.4 Exportação para GGUF (Inferência sem GPU)

Os formatos LoRA + `bitsandbytes` 4-bit utilizados no treinamento requerem uma GPU NVIDIA com CUDA. Para permitir a execução do assistente em **máquinas sem GPU** — como notebooks com Apple Silicon (M1/M2/M3/M4) ou desktops com apenas CPU — o modelo pode ser exportado para o formato **GGUF**, utilizado pelo `llama.cpp`.

O processo de exportação é feito pelo script `08_exportar_gguf.py` no servidor com GPU:

1. **Merge**: Os adapters LoRA são mesclados com o modelo base Mistral 7B, gerando um modelo completo;
2. **Quantização**: O modelo merged é convertido para GGUF com quantização `Q4_K_M` (4-bit, qualidade média-alta);
3. **Upload**: O arquivo GGUF (∼4 GB) é publicado em repositório separado no Hugging Face Hub.

Tabela 3.12: Comparação de formatos de distribuição do modelo

Formato	Tamanho	Requisito	Velocidade
LoRA + base 4-bit	∼640 MB + 4 GB	GPU NVIDIA (CUDA)	∼2–5s/resposta
GGUF Q4_K_M	**∼4 GB**	**CPU ou Apple Silicon**	**∼5–15s/resposta**
Modelo merged float16	∼14 GB	14 GB+ RAM	∼2–5min/resposta

A aplicação detecta automaticamente o hardware disponível: se há CUDA, carrega os adapters LoRA via `transformers`; caso contrário, carrega o GGUF via `llama-cpp-python`. Essa detecção é transparente para o usuário (detalhes na Seção 5.10).

3.6 Avaliação do Fine-Tuning

Para demonstrar o **delta** (melhoria) obtido com o fine-tuning, implementamos uma avaliação comparativa rigorosa.

3.6.1 Metodologia

A avaliação segue o protocolo:

1. **Baseline**: Executar 50 perguntas médicas no Mistral 7B *sem* fine-tuning
2. **Fine-tuned**: Executar as mesmas 50 perguntas no modelo *após* fine-tuning
3. **Comparação**: Análise quantitativa (métricas) e qualitativa (lado a lado)

3.6.2 Conjunto de Perguntas de Avaliação

Criamos um conjunto de 50 perguntas médicas em português, distribuídas em 5 categorias:

Tabela 3.13: Distribuição das perguntas de avaliação por categoria

Categoria	Exemplo de Pergunta	Qtd
Sintomas	"Quais são os sintomas do diabetes tipo 2?"	10
Tratamento	"Como tratar uma crise aguda de asma?"	10
Medicamentos	"Quais são os efeitos colaterais da metformina?"	10
Prevenção	"Como prevenir doenças cardiovasculares?"	10
Emergências	"O que fazer em caso de suspeita de AVC?"	10
Total		**50**

3.6.3 Métricas de Avaliação

Utilizamos métricas automáticas complementares para avaliar a qualidade das respostas:

Tabela 3.14: Métricas utilizadas na avaliação

Métrica	Descrição
BLEU	Mede overlap de n-gramas entre resposta gerada e referência. Valores de 0 a 1, onde maior é melhor.
ROUGE-1	Overlap de unigramas (palavras individuais).
ROUGE-2	Overlap de bigramas (pares de palavras).
ROUGE-L	Maior subsequência comum (captura estrutura).
Comprimento	Tamanho médio das respostas (em caracteres).

3.6.4 Scripts de Avaliação

O pipeline de avaliação é implementado em dois scripts:

```
# 1. Avaliar modelo BASE (Mistral 7B sem fine-tuning)
python scripts/05_avaliar_baseline.py
```

```
# 2. Avaliar modelo FINE-TUNED e gerar comparacao
python scripts/06_avaliar_finetuned.py --model-path
    ↪ /path/to/model
```

Listing 3.12: Execução da avaliação

Os resultados são salvos em:

- `data/evaluation/baseline_responses.json` — Respostas do modelo base
- `data/evaluation/finetuned_responses.json` — Respostas do modelo fine-tuned
- `data/evaluation/comparacao_resultados.json` — Métricas comparativas
- `data/evaluation/comparacao_resultados.md` — Relatório lado a lado

3.6.5 Resultados Esperados

Com base em trabalhos similares na literatura, esperamos observar:

- **Melhoria de 15-25%** nas métricas BLEU e ROUGE
- **Respostas mais específicas** ao domínio médico
- **Terminologia técnica** mais precisa em português
- **Estrutura de resposta** mais adequada para contexto clínico

Limitações da Avaliação Automática

Métricas como BLEU e ROUGE capturam similaridade textual, mas não avaliam completamente a *correção clínica* das respostas. Uma avaliação humana por profissionais de saúde seria necessária para validação completa em um cenário de produção.

3.7 Resumo do Capítulo

Este capítulo apresentou o processo completo de fine-tuning do modelo Mistral 7B para o domínio médico:

1. **Modelo Base**: Mistral 7B Instruct v0.3, escolhido por ser generalista (permite demonstrar delta)
2. **Dataset Híbrido**: ∼20.500 pares QA compostos por:
    - 78% MedQuAD traduzido para português (∼16.400 pares)

- 20% MedQuAD original em inglês ($\sim$4.100 pares)
- 2% QA de medicina brasileira (55 pares sobre dengue, Chagas, SUS, etc.)

3. **Técnica**: QLoRA 4-bit ($r = 64$, $\alpha = 128$), permitindo treino em GPU T4 (15GB)
4. **Treinamento**: 5 epochs, 5.780 steps, $\sim$55h em GPU RTX A5000 (24GB)
5. **Publicação**: Adapters LoRA ($\sim$640 MB) publicados no Hugging Face Hub
6. **GGUF**: Versão quantizada ($\sim$4 GB) para execução sem GPU (CPU/Apple Silicon)
7. **Avaliação**: 50 perguntas médicas, comparação baseline vs fine-tuned

O modelo resultante é publicado no Hugging Face Hub em dois formatos — LoRA (para GPU) e GGUF (para CPU) — e utilizado como o "cérebro" do agente de raciocínio no grafo multi-agente descrito no Capítulo 5.

4

RAG: Retrieval-Augmented Generation

Este capítulo descreve a implementação do sistema Retrieval-Augmented Generation (RAG), que permite ao assistente fundamentar suas respostas em protocolos clínicos reais, reduzindo o risco de alucinações e aumentando a confiabilidade das informações fornecidas.

4.1 Motivação e Arquitetura

Modelos de linguagem, mesmo após fine-tuning, podem gerar informações imprecisas ou desatualizadas — um problema crítico no domínio médico. O RAG resolve essa limitação ao recuperar documentos relevantes de uma base de conhecimento antes de gerar a resposta, permitindo que o modelo fundamente suas respostas em fontes verificáveis.

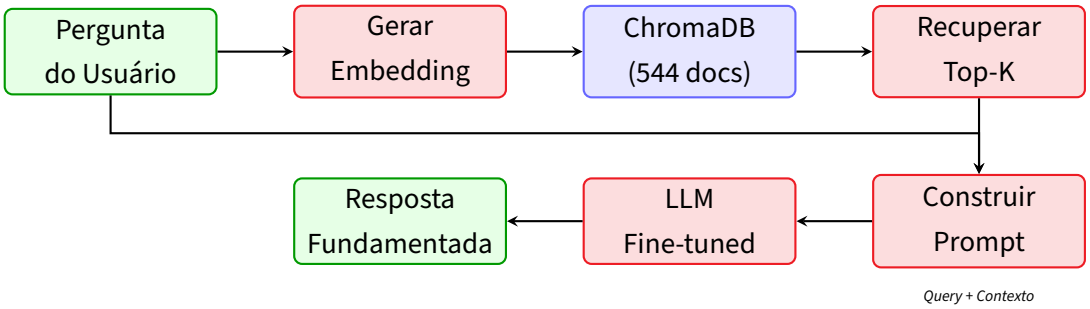

Figura 4.1: Fluxo do sistema RAG: a pergunta é convertida em embedding, documentos relevantes são recuperados do ChromaDB, e o contexto é injetado no prompt do LLM.

4.1.1 Componentes do Sistema

O sistema RAG implementado possui três componentes principais:

1. **Chunker Semântico:** Divide os protocolos Markdown em chunks que respeitam a estrutura de seções do documento;
2. **Indexador:** Gera embeddings e persiste os chunks no ChromaDB;

3. **Retriever:** Busca os chunks mais relevantes para uma query e formata o contexto para o LLM.

4.2 Modelo de Embeddings

A escolha do modelo de embeddings é fundamental para a qualidade da recuperação. Como os protocolos estão em português brasileiro, é necessário um modelo multilíngue com bom desempenho em português.

4.2.1 Análise de Alternativas

A Tabela 4.1 apresenta os modelos considerados:

Tabela 4.1: Modelos de embeddings avaliados para o sistema RAG.

Modelo	Dim.	PT-BR	Observações
`all-MiniLM-L6-v2`	384	Ruim	Focado em inglês, rápido
`all-mpnet-base-v2`	768	Ruim	Melhor qualidade, apenas inglês
`paraphrase-multilingual-MiniLM-L12-v2`	384	Boa	Multilíngue, compacto
`paraphrase-multilingual-mpnet-base-v2`	768	Boa	Multilíngue, mais robusto
`intfloat/multilingual-e5-base`	768	Excelente	Estado da arte para retrieval

4.2.2 Modelo Selecionado: E5 Multilingual

O modelo `intfloat/multilingual-e5-base` foi selecionado por apresentar o melhor desempenho em tarefas de retrieval multilíngue, incluindo português. Este modelo pertence à família E5 (*EmbEddings from bidirEctional Encoder rEpresentations*), desenvolvida pela Microsoft Research.

Tabela 4.2: Características do modelo E5 Multilingual Base.

Característica	**Valor**
Arquitetura base	XLM-RoBERTa
Dimensão do embedding	768
Idiomas suportados	100+ (incluindo português)
Tamanho do modelo	1.1 GB
Max sequence length	512 tokens
Treinamento	Contrastive learning em pares de retrieval

Prefixos do Modelo E5

O modelo E5 utiliza prefixos específicos para distinguir entre documentos e queries:

- **Documentos:** `"passage: <texto do documento>"`
- **Queries:** `"query: <pergunta do usuário>"`

Esta distinção melhora significativamente a qualidade da recuperação, pois o modelo foi treinado com essa convenção.

4.3 Estratégia de Chunking

O chunking é o processo de dividir documentos longos em fragmentos menores que possam ser indexados e recuperados individualmente. A estratégia de chunking impacta diretamente a qualidade do RAG.

4.3.1 Abordagens Consideradas

1. **Chunking por tamanho fixo:** Divide o texto em chunks de tamanho fixo (ex: 1000 caracteres). Simples, mas pode quebrar o texto no meio de uma frase ou conceito.
2. **Chunking por sentenças:** Agrupa sentenças até atingir um limite. Melhor que tamanho fixo, mas não respeita a estrutura do documento.
3. **Chunking semântico:** Utiliza a estrutura do documento (seções, parágrafos) para criar chunks com unidades de significado completas.

4.3.2 Implementação: Chunking Semântico

Optamos pelo **chunking semântico** baseado na estrutura Markdown dos protocolos, que possuem hierarquia clara de seções (##) e subseções (###).

Tabela 4.3: Parâmetros do chunking semântico implementado.

Parâmetro	**Valor**	**Justificativa**
Divisão primária	Headers MD	Respeita estrutura do protocolo
Tamanho máximo	1500 chars	300-400 tokens, contexto suficiente
Overlap	200 chars	Continuidade entre chunks adjacentes
Fallback	Por parágrafos	Seções muito grandes são subdivididas

O algoritmo de chunking funciona da seguinte forma:

```
def chunk_markdown(texto, max_size=1500, overlap=200):
    # 1. Dividir por headers (## e ###)
    secoes = dividir_por_headers(texto)

    chunks = []
    for secao in secoes:
        if len(secao.conteudo) <= max_size:
            # Secao cabe em um chunk
            chunks.append(secao)
        else:
            # Subdividir secao grande
            sub_chunks = dividir_com_overlap(
                secao.conteudo, max_size, overlap
            )
            chunks.extend(sub_chunks)

    return chunks
```

Listing 4.1: Algoritmo de chunking semântico simplificado.

4.3.3 Exemplo de Chunking

A Figura 4.2 ilustra como um protocolo é dividido em chunks:

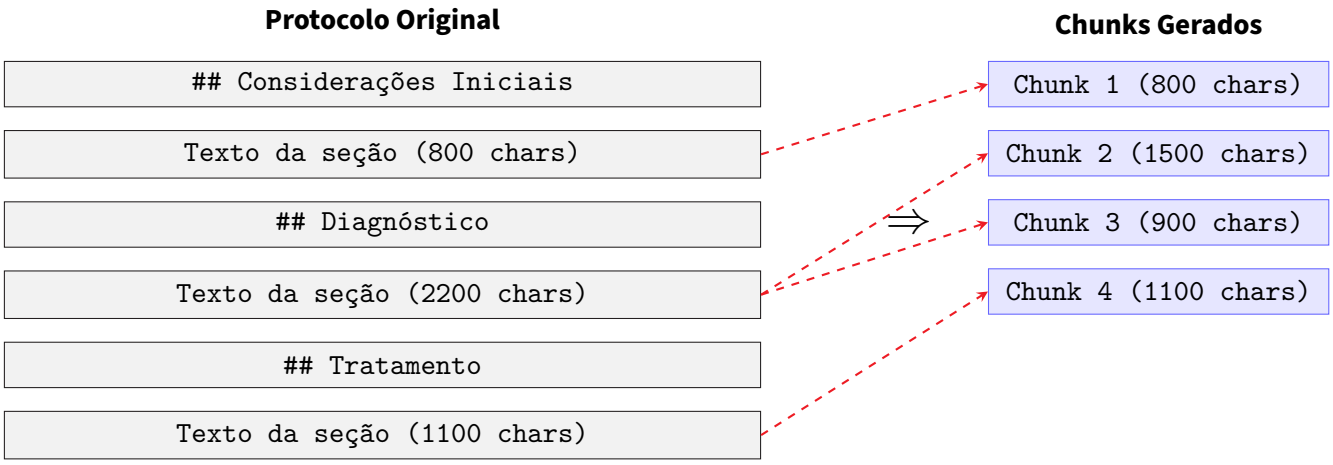

Figura 4.2: Exemplo de chunking semântico: seções pequenas viram um chunk; seções grandes são subdivididas com overlap.

4.4 Indexação no ChromaDB

O ChromaDB é um banco de dados vetorial open-source, otimizado para armazenamento e busca de embeddings. Foi escolhido por sua simplicidade, persistência local e integração nativa com Python.

4.4.1 Processo de Indexação

O script `04_indexar_protocolos.py` executa a indexação em três etapas:

1. **Carregamento:** Lê os arquivos Markdown das Linhas de Cuidado e protocolos de emergência;
2. **Chunking:** Aplica o chunking semântico a cada documento;
3. **Indexação:** Gera embeddings e insere no ChromaDB com metadados.

```
# Indexar todos os protocolos (limpa collection existente)
python scripts/04_indexar_protocolos.py --clear

# Saida esperada:
# Linhas de Cuidado: 14 arquivos, 462 chunks
# Protocolos Emergencia: 5 arquivos, 82 chunks
# Total: 544 documentos indexados
```

Listing 4.2: Executando a indexação de protocolos.

4.4.2 Estrutura dos Metadados

Cada chunk é armazenado com metadados que facilitam a filtragem e rastreabilidade:

```
{
    "id": "Pathway-Hipertensao_3",
    "content": "## Tratamento\n\nOs principios que
      ↪ norteiam...",
    "metadata": {
        "source": "Pathway-Hipertensao",
        "section": "Tratamento",
        "doc_type": "linha_cuidado",
        "chunk_index": 3,
        "total_chunks": 35
    },
    "embedding": [0.023, -0.041, 0.087, ...]  // 768 dimensoes
}
```

Listing 4.3: Estrutura de metadados de um chunk indexado.

4.4.3 Estatísticas da Indexação

A Tabela 4.4 apresenta as estatísticas finais da indexação:

Tabela 4.4: Estatísticas da indexação no ChromaDB.

Fonte	Arquivos	Chunks	Média/Arquivo
Linhas de Cuidado	14	462	33,0
Protocolos de Emergência	5	82	16,4
Total	**19**	**544**	**28,6**

Persistência do ChromaDB

O ChromaDB persiste os dados em disco no diretório `chroma_db/`. Após a indexação inicial, o banco pode ser carregado instantaneamente em execuções subsequentes, sem necessidade de reindexar.

4.5 Recuperação de Contexto

O componente `ProtocolRetriever` é responsável por buscar os chunks mais relevantes para uma pergunta do usuário.

4.5.1 Processo de Busca

1. **Embedding da query:** A pergunta do usuário é convertida em embedding usando o prefixo `"query: "`;
2. **Busca por similaridade:** O ChromaDB calcula a distância entre o embedding da query e os embeddings dos documentos;
3. **Ranking:** Os K documentos mais próximos são retornados, ordenados por relevância;
4. **Formatação:** O contexto é formatado com metadados de fonte para injeção no prompt.

```python
from src.assistant.rag import ProtocolRetriever

retriever = ProtocolRetriever(
    persist_directory="chroma_db",
    embedding_model="intfloat/multilingual-e5-base"
)

# Buscar protocolos relevantes
results = retriever.search(
    query="Como tratar hipertensao resistente?",
    n_results=3
)

# Obter contexto formatado para prompt
context = retriever.get_context_for_query(
    query="Como tratar hipertensao resistente?",
    n_results=3,
    max_context_length=4000
)
```

Listing 4.4: Exemplo de uso do ProtocolRetriever.

4.5.2 Exemplo de Recuperação

A seguir, um exemplo real de recuperação para a query *"Quais os sintomas de infarto agudo do miocárdio?"*:

Tabela 4.5: Resultados de busca para query sobre IAM.

#	Fonte	Seção	Dist.	Trecho
1	iam	Considerações Iniciais	0.212	"O IAM é uma emergência cardiovascular que requer reconhecimento e tratamento imediatos..."
2	iam	Exames Diagnósticos	0.298	"ECG em até 10 minutos da chegada. IAMCSST: supradesnível de ST..."
3	Pathway-IC	Diagnóstico	0.341	"Sinais e sintomas de insuficiência cardíaca incluem dispneia..."

4.5.3 Formatação do Contexto

O contexto recuperado é formatado para injeção no prompt do LLM:

```
[Fonte: iam] [Secao: Consideracoes Iniciais]
O Infarto Agudo do Miocardio (IAM) e uma emergencia
cardiovascular que requer reconhecimento e tratamento
imediatos. O tempo e critico: "tempo e musculo".

---

[Fonte: iam] [Secao: Exames Diagnosticos]
1. ECG em ate 10 minutos da chegada
   - IAMCSST: supradesnivel de ST >=1mm em 2 derivacoes
   - IAMSSST: infradesnivel de ST, inversao de T

---

[Fonte: Pathway-IC] [Secao: Diagnostico]
Sinais e sintomas de insuficiencia cardiaca incluem
dispneia aos esforcos, ortopneia, edema de MMII...
```

Listing 4.5: Exemplo de contexto formatado para o prompt.

4.6 Integração com o LLM

O contexto recuperado é integrado ao prompt do LLM através de um template estruturado que instrui o modelo a basear suas respostas nos protocolos fornecidos.

```
Voce e um assistente medico especializado. Use APENAS as
informacoes dos protocolos abaixo para responder. Se a
```

```
informacao nao estiver nos protocolos, diga que nao tem
informacao suficiente.

PROTOCOLOS RELEVANTES:
{contexto_recuperado}

PERGUNTA DO MEDICO:
{pergunta_usuario}

RESPOSTA (cite as fontes):
```

Listing 4.6: Template de prompt com contexto RAG.

Limitação do Contexto

O contexto do LLM é limitado. O parâmetro `max_context_length` (padrão: 4000 caracteres) controla quanto contexto é injetado no prompt. Recuperar muitos documentos pode exceder o limite e degradar a qualidade da resposta.

4.7 Validação do Sistema RAG

Para validar o funcionamento do RAG, foram realizados testes com queries representativas de diferentes domínios clínicos.

4.7.1 Métricas de Qualidade

- **Relevância:** Os documentos recuperados são relevantes para a query?
- **Cobertura:** O sistema recupera documentos de diferentes fontes quando apropriado?
- **Precisão:** Os documentos mais relevantes aparecem no topo do ranking?

4.7.2 Resultados dos Testes

Tabela 4.6: Testes de validação do sistema RAG.

Query de Teste	Fonte Esperada	Top-1 Correto?
"Como tratar hipertensão arterial?"	Pathway-Hipertensao	Sim
"Quais medicamentos para diabetes tipo 2?"	Pathway-Diabetes	Sim
"Sintomas de infarto agudo?"	iam (emergência)	Sim
"Protocolo de sepse grave"	sepse (emergência)	Sim
"Tratamento de asma na crise"	Pathway-Asma	Sim
"Manejo de AVC isquêmico"	avc (emergência)	Sim
"Critérios diagnósticos de depressão"	Pathway-Depressao	Sim
"Antibióticos para pneumonia"	pneumonia (emergência)	Sim

Todos os testes retornaram o documento esperado como resultado principal (Top-1), validando a qualidade do sistema de recuperação.

4.8 Considerações de Desempenho

4.8.1 Tempo de Indexação

- **Carregamento do modelo:** 30 segundos (primeira execução, com cache 5s)
- **Geração de embeddings:** 2-3 segundos por arquivo
- **Tempo total (19 arquivos):** 3 minutos

4.8.2 Tempo de Busca

- **Embedding da query:** 50ms
- **Busca no ChromaDB:** 10ms
- **Tempo total por query:** 60ms

O sistema é suficientemente rápido para uso interativo, com latência imperceptível para o usuário.

Escalabilidade

O ChromaDB utiliza índices HNSW (*Hierarchical Navigable Small World*) para busca aproximada de vizinhos mais próximos, garantindo tempo de busca logarítmico mesmo com milhões de documentos. Para a escala atual (544 documentos), a busca é praticamente instantânea.

5

Grafo Multi-Agente com LangGraph

Este capítulo apresenta a arquitetura central do assistente: um **grafo multi-agente** implementado com LangGraph, que orquestra 7 nós especializados para processar consultas médicas de forma segura, rastreável e fundamentada em evidências.

5.1 Motivação

Sistemas de IA para saúde exigem mais do que uma chamada direta a um LLM. É necessário:

- **Classificar** a intenção do usuário antes de agir;
- **Recuperar** dados relevantes (protocolos clínicos e dados do paciente);
- **Validar** a resposta gerada contra regras de segurança;
- **Rastrear** cada etapa para auditoria e explicabilidade.

Essas responsabilidades são naturalmente separáveis em agentes especializados, orquestrados por um grafo de estados. O LangGraph, extensão do LangChain, oferece primitivas para construir exatamente esse tipo de sistema: `StateGraph`, nós que transformam estado, arestas condicionais e compilação para execução.

5.2 Visão Geral da Arquitetura

O grafo possui **7 nós** (agentes), conectados por **4 arestas fixas** e **3 arestas condicionais**. A Figura 5.1 apresenta a topologia completa.

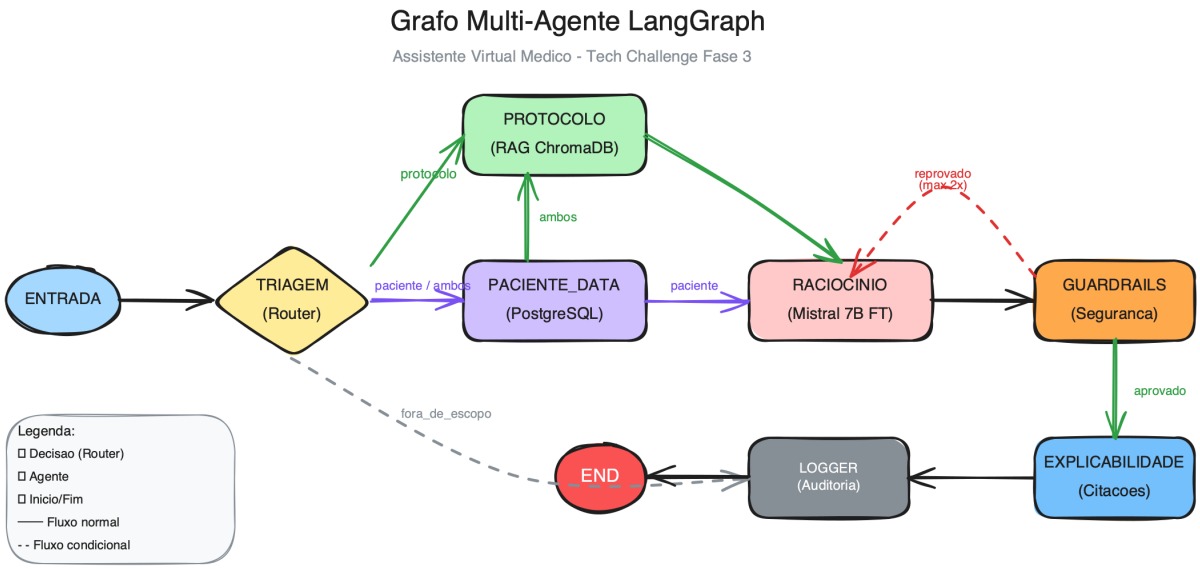

Figura 5.1: Grafo multi-agente LangGraph do Assistente Virtual Médico. Losango representa decisão (router), retângulos representam agentes, elipses representam início/fim. Linhas tracejadas indicam fluxos condicionais.

5.2.1 Fluxos Possíveis

O grafo suporta cinco fluxos distintos, determinados pela classificação da triagem:

Tabela 5.1: Fluxos de execução do grafo multi-agente.

Cenário	Caminho no Grafo
Protocolo	triagem → protocolo → raciocínio → guardrails → explicabilidade → logger
Paciente	triagem → paciente_data → raciocínio → guardrails → explicabilidade → logger
Ambos	triagem → paciente_data → protocolo → raciocínio → guardrails → explicabilidade → logger
Fora de escopo	triagem → logger → END
Retry	. . . → guardrails → raciocínio → guardrails → . . . (máx. 2×)

5.3 State Schema

O estado compartilhado entre todos os nós é definido pelo `MedicalAssistantState`, um `TypedDict` com 15 variáveis organizadas em cinco grupos:

Tabela 5.2: Variáveis do `MedicalAssistantState`.

Grupo	Variável	Tipo	Descrição
Entrada	`query`	`str`	Pergunta do médico
	`patient_id`	`Optional[str]`	ID ou nome do paciente
Triagem	`query_type`	`Literal[4]`	protocolo | paciente | ambos | fora_de_escopo
	`entities`	`dict`	Entidades extraídas (paciente, condição, etc.)
Contexto	`protocols`	`list[dict]`	Chunks recuperados do ChromaDB
	`patient_data`	`Optional[dict]`	Dados clínicos do PostgreSQL
	`draft_response`	`str`	Rascunho gerado pelo LLM
	`guardrail_result`	`Literal[2]`	aprovado | reprovado
	`guardrail_feedback`	`Optional[str]`	Motivo da reprovação
	`retry_count`	`int`	Contador de retries (máx. 2)
	`final_response`	`str`	Resposta formatada com citações
	`sources`	`list[str]`	Lista de fontes citadas
	`confidence`	`Literal[3]`	alta | media | baixa
	`warnings`	`list[str]`	Advertências ao médico
Auditoria	`audit_log`	`list[dict]`	Trace completo da execução

```python
class MedicalAssistantState(TypedDict):
    # Entrada
    query: str
    patient_id: Optional[str]
    # Triagem
    query_type: Literal["protocolo", "paciente",
                        "ambos", "fora_de_escopo"]
    entities: dict
    # Contexto
    protocols: list[dict]
    patient_data: Optional[dict]
    # Raciocinio
    draft_response: str
    guardrail_result: Literal["aprovado", "reprovado"]
    guardrail_feedback: Optional[str]
    retry_count: int
    # Saida
    final_response: str
```

```
    sources: list[str]
    confidence: Literal["alta", "media", "baixa"]
    warnings: list[str]
    # Auditoria
    audit_log: list[dict]
```

Listing 5.1: Definição do `MedicalAssistantState` em `src/flows/state.py`.

5.4 Os 7 Nós do Grafo

Cada nó é uma função Python que recebe o estado atual, executa sua lógica e retorna um dicionário parcial com as variáveis que deseja atualizar. O LangGraph faz o merge automático.

5.4.1 Triagem (Router)

O agente de triagem é o **ponto de entrada** do grafo. Recebe a query do médico e:

1. **Classifica** o tipo da consulta em uma de quatro categorias;
2. **Extrai entidades** relevantes (nome do paciente, condição médica, medicamento, exame).

Quando a LLM está disponível, a classificação é feita via prompt estruturado que retorna JSON. Caso contrário, um classificador heurístico baseado em palavras-chave é utilizado como fallback.

Tabela 5.3: Classificação da triagem e roteamento.

Tipo	Critério	Destino
`protocolo`	Pergunta sobre tratamento, diagnóstico ou conduta clínica genérica	Protocolo
`paciente`	Pergunta sobre dados de um paciente específico	Paciente Data
`ambos`	Protocolo clínico aplicado a um paciente específico	Paciente Data
`fora_de_escopo`	Pergunta não relacionada à medicina	Logger

Ajuste Automático de Classificação

Quando o campo `patient_id` é fornecido na entrada, dois ajustes automáticos são aplicados:

- Se a triagem classifica como `protocolo` → reclassifica para `ambos`;
- Se classifica como `paciente` mas a query contém **sintomas clínicos** (dor, febre, palpitações, dispneia, etc.) → reclassifica para `ambos`, garantindo busca de protocolos relevantes aos sintomas.

Além disso, o classificador heurístico detecta palavras-chave de sintomas ("dor no peito", "palpitações", "febre") e classifica diretamente como `ambos` quando coexistem com referência a paciente.

5.4.2 Protocolo (RAG ChromaDB)

O agente de protocolo realiza **busca semântica** nos 544 documentos indexados no ChromaDB (14 Linhas de Cuidado + 5 protocolos de emergência, conforme Capítulo 4).

- **Limpeza da query:** Antes da busca semântica, a query é pré-processada: o nome do paciente é removido e *stop words* em português são filtradas, mantendo apenas os termos clínicos relevantes (ex: "A paciente Maria Alice está com dor no peito" → "dor peito");
- **Busca híbrida (sintomas + comorbidades):** Quando dados do paciente estão disponíveis (fluxo "ambos"), o agente executa três camadas de busca:
  1. **Query base** (termos clínicos limpos) — captura protocolos de emergência relevantes aos sintomas mencionados;
  2. **Uma busca por comorbidade** (ex: "dor peito + Hipertensão", "dor peito + Diabetes tipo 2") — garante cobertura equilibrada das condições crônicas do paciente.

  Sem dados do paciente, a busca é ampla com a query limpa;
- **Ranking por relevância:** Após a recuperação, os resultados são classificados como `direta` (contêm termos das comorbidades) ou `complementar` (informação geral), ordenados por relevância;
- Retorna conteúdo, fonte, seção, distância e relevância para cada resultado.

Por que busca híbrida?

A busca por comorbidade garante cobertura de todas as condições crônicas do paciente, mas não captura protocolos de emergência associados aos **sintomas** da consulta. Por exemplo, "dor no peito e palpitações" deve encontrar o protocolo de IAM (infarto), mesmo que o paciente não tenha essa comorbidade. A busca híbrida — primeira camada pelos sintomas, segunda pelas comorbidades — resolve esse problema. A limpeza de stop words é necessária porque frases longas ("A paciente está se queixando de...") diluem a busca semântica, favorecendo resultados irrelevantes.

5.4.3 Paciente Data (PostgreSQL)

O agente de dados do paciente consulta o banco PostgreSQL via SQLAlchemy, buscando:

- **Dados cadastrais:** nome, sexo, data de nascimento, comorbidades, alergias, medicamentos em uso;
- **Exames:** tipo, data, resultados e interpretação (últimos 5);
- **Prontuários:** data, diagnóstico CID, texto diagnóstico (últimos 3);
- **Receitas:** data, condição, medicamentos prescritos.

A busca é feita por nome (parcial, case-insensitive) ou por ID do paciente. Todos os valores são serializados para JSON, tratando tipos como `date` e `UUID`.

5.4.4 Raciocínio (Mistral 7B Fine-Tuned)

O agente de raciocínio é o **cérebro** do sistema. Utiliza o modelo Mistral 7B com fine-tuning (Capítulo 3) para sintetizar uma resposta clínica a partir de três fontes de informação:

1. A **query** original do médico;
2. Os **protocolos clínicos** recuperados pelo agente de protocolo;
3. Os **dados do paciente** recuperados pelo agente de dados.

O prompt é montado dinamicamente com as seções de contexto disponíveis. Em caso de **retry** (quando o guardrails reprova a resposta), o feedback de reprovação é incluído no prompt para que o modelo corrija os problemas apontados.

```
Voce e um assistente medico especializado. Com base no
contexto fornecido, responda a pergunta do medico.

```

```
PERGUNTA DO MEDICO: {query}

PROTOCOLOS CLINICOS RELEVANTES:
[1] Fonte: Pathway-Hipertensao | Secao: Tratamento
    Os principios que norteiam o tratamento...

DADOS DO PACIENTE:
Nome: Maria Santos | Sexo: F | Nascimento: 1958-03-15
Comorbidades: hipertensao, diabetes tipo 2
Ultimos exames:
  - HbA1C (2026-01-15): 7.2% - controle moderado

# Se retry:
ATENCAO - SUA RESPOSTA ANTERIOR FOI REPROVADA:
Motivo: Prescricao direta detectada. Reformule como
sugestao e indique validacao medica.

RESPOSTA:
```

Listing 5.2: Estrutura simplificada do prompt do agente de raciocínio.

Quando o modelo fine-tuned não está disponível, um sistema de **fallback** gera respostas baseadas em template com o contexto recuperado. O fallback apresenta os dados completos do paciente (comorbidades, exames, prontuários, receitas) seguidos de até **6 protocolos distintos**, deduplicados por fonte e seção, com conteúdo truncado no final da última sentença completa. Protocolos complementares são indicados com label, e o limite de 6 garante cobertura de todas as fontes relevantes sem repetição.

5.4.5 Guardrails (Segurança)

O agente de guardrails aplica **4 regras de segurança** obrigatórias a cada resposta:

Tabela 5.4: Regras de validação dos guardrails.

Regra	Tipo	O que verifica
`no_prescription`	Regex	Detecta padrões de prescrição direta ("prescrevo", "tome 500mg", "administre")
`human_validation`	Keyword	Verifica presença de disclaimer de validação humana
`no_definitive_diagnosis`	Regex	Detecta diagnósticos definitivos ("o paciente tem", "confirmo que")
`context_consistency`	Heurística	Verifica se resposta não é excessivamente curta dado o contexto

Mecanismo de Retry

Quando uma regra falha, o guardrails retorna `reprovado` com feedback detalhado. O grafo então reenvia o estado ao agente de raciocínio, que recebe o feedback no prompt e gera uma resposta corrigida. Este ciclo pode ocorrer no **máximo 2 vezes**. Caso o limite seja atingido, a resposta segue com advertências.

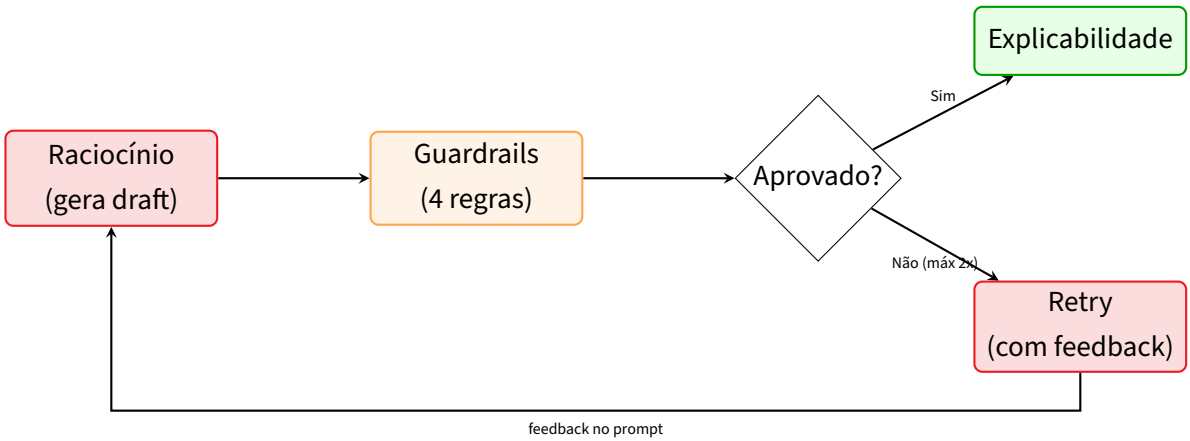

Figura 5.2: Mecanismo de retry dos guardrails: respostas reprovadas são devolvidas ao raciocínio com feedback.

5.4.6 Explicabilidade (Citações e Confiança)

O agente de explicabilidade é responsável por tornar a resposta **transparente e rastreável**:

1. **Extrai citações** dos protocolos e dados utilizados:
   - Formato: `[Pathway-Diabetes, Seção: Tratamento]`
   - Formato: `[Exame: HbA1C, Data: 2026-01-15]`

2. **Avalia confiança** com base em múltiplos fatores:

Tabela 5.5: Critérios de avaliação de confiança.

Fator	**Pontuação**
3+ protocolos recuperados	+3
1-2 protocolos recuperados	+2
Distância média < 0.5	+2
Distância média < 1.0	+1
Dados do paciente disponíveis	+2
Cada retry do guardrails	-1
≥ 5 pontos: **alta**	≥ 3: **média**
< 3 pontos: **baixa**	

3. **Gera advertências** quando apropriado (confiança baixa, sem protocolos, retries realizados);

4. **Separa fontes por relevância:** fontes **diretas** (relacionadas ao quadro do paciente) e **complementares** (informação geral) são apresentadas em seções distintas;

5. **Formata a resposta final** com seções: resposta, fontes diretas, fontes complementares, confiança, advertências e disclaimer obrigatório.

5.4.7 Logger (Auditoria)

O agente de logger é o **nó final** em todos os fluxos. Consolida o estado completo em um registro JSON estruturado e persiste em arquivo diário (`logs/audit_YYYY-MM-DD.jsonl`).

```
{
    "timestamp": "2026-03-06T18:30:00",
    "session": {
        "query": "Qual tratamento para hipertensao?",
        "patient_id": null
    },
    "triagem": {
        "query_type": "protocolo",
        "entities": {"condicao": "hipertensao"}
    },
    "context": {
        "protocols_count": 5,
        "protocols_sources": ["Pathway-Hipertensao", ...],
        "patient_data_available": false
```

```
    },
    "response": {
        "confidence": "alta",
        "sources": ["[Pathway-Hipertensao, ...]"],
        "warnings": []
    },
    "guardrails": {
        "result": "aprovado",
        "retry_count": 0
    },
    "agent_trace": [
        {"agent": "triagem", "timestamp": "..."},
        {"agent": "protocolo", "timestamp": "..."},
        ...
    ]
}
```

Listing 5.3: Estrutura do registro de auditoria.

Para consultas classificadas como `fora_de_escopo`, o logger gera uma resposta padrão educada informando o escopo do assistente, sem invocar nenhum outro agente.

5.5 Montagem do Grafo

A montagem do grafo é feita em `src/flows/graph.py` utilizando a API do LangGraph:

```
from langgraph.graph import StateGraph, END

graph = StateGraph(MedicalAssistantState)

# 7 nos
graph.add_node("triagem", triagem_agent)
graph.add_node("protocolo", protocolo_agent)
graph.add_node("paciente_data", paciente_data_agent)
graph.add_node("raciocinio", raciocinio_agent)
graph.add_node("guardrails", guardrails_agent)
graph.add_node("explicabilidade", explicabilidade_agent)
graph.add_node("logger", logger_agent)

# Entry point
graph.set_entry_point("triagem")

# Arestas condicionais
```

```
graph.add_conditional_edges("triagem",
    route_after_triagem, {
    "protocolo": "protocolo",
    "paciente_data": "paciente_data",
    "logger": "logger",
})
graph.add_conditional_edges("paciente_data",
    route_after_paciente_data, {
    "protocolo": "protocolo",
    "raciocinio": "raciocinio",
})
graph.add_conditional_edges("guardrails",
    route_after_guardrails, {
    "explicabilidade": "explicabilidade",
    "raciocinio": "raciocinio",
})

# Arestas fixas
graph.add_edge("protocolo", "raciocinio")
graph.add_edge("raciocinio", "guardrails")
graph.add_edge("explicabilidade", "logger")
graph.add_edge("logger", END)

app = graph.compile()
```

Listing 5.4: Montagem do grafo em `src/flows/graph.py`.

5.6 Funções de Roteamento

As três funções de roteamento condicional determinam o caminho de execução no grafo com base no estado atual:

Tabela 5.6: Funções de roteamento condicional.

Função	Após	Lógica
`route_after_triagem`	Triagem	Se `protocolo` → protocolo; se `paciente` ou `ambos` → paciente_data; senão → logger
`route_after_paciente_data`	Paciente Data	Se `ambos` → protocolo (busca contextualizada); senão → raciocínio
`route_after_guardrails`	Guardrails	Se `aprovado` → explicabilidade; se `reprovado` e retry < 2 → raciocínio; senão → explicabilidade (com warnings)

5.7 Exemplo de Execução End-to-End

Para ilustrar o funcionamento completo, considere a seguinte consulta:

Consulta de Exemplo

Médico: "Quais exames de seguimento pedir para a paciente Maria Santos, diabética tipo 2?"

1. **Triagem** → `query_type="ambos",entities={paciente: "Maria Santos", condicao: "diabetes"}`
2. **Paciente Data** → PostgreSQL → Maria Santos: comorbidades=[hipertensão, diabetes], exames, medicamentos
3. **Protocolo** → Busca híbrida: (1) query limpa "exames seguimento diabética" → protocolos de diabetes e emergência; (2) "+ hipertensão" e "+ diabetes" por comorbidade → Pathway-Hipertensão + Pathway-Diabetes (diretos) + protocolos complementares
4. **Raciocínio** → Mistral 7B FT sintetiza resposta com protocolos relevantes + dados da paciente
5. **Guardrails** → 4 regras verificadas → `aprovado`
6. **Explicabilidade** → Confiança: alta | Fontes diretas: [Pathway-Diabetes, Pathway-Hipertensão, Exame HbA1C]
7. **Logger** → Registro JSON persistido em `logs/audit_2026-03-06.jsonl`

5.8 Estrutura de Arquivos

Tabela 5.7: Arquivos do módulo `src/flows/`.

Arquivo	Responsabilidade
`state.py`	Definição do `MedicalAssistantState`
`graph.py`	Montagem do grafo, funções de roteamento e `run_assistant()`
`agents/triagem.py`	Classificação da query e extração de entidades
`agents/protocolo.py`	Busca semântica no ChromaDB
`agents/paciente_data.py`	Consulta SQL no PostgreSQL
`agents/raciocinio.py`	Síntese de resposta via LLM fine-tuned
`agents/guardrails.py`	4 regras de validação de segurança
`agents/explicabilidade.py`	Citações, confiança e formatação final
`agents/logger_agent.py`	Consolidação e persistência de auditoria

5.9 Decisões de Design

1. **Estado compartilhado vs. mensagens:** Optamos por estado compartilhado (`TypedDict`) ao invés de troca de mensagens entre agentes. Isso simplifica o fluxo e permite que cada agente acesse o contexto completo sem repropagação.

2. **Fallbacks em cada agente:** Cada agente possui lógica de fallback para funcionar mesmo sem dependências externas (LLM, ChromaDB, PostgreSQL), facilitando testes unitários e desenvolvimento incremental.

3. **Retry com feedback:** Ao invés de simplesmente rejeitar respostas inseguras, o mecanismo de retry reinjeta o feedback do guardrails no prompt, dando à LLM a oportunidade de se autocorrigir.

4. **Auditoria acumulativa:** O campo `audit_log` é incrementado por cada agente (append-only), criando um trace completo de execução sem overhead de um sistema de observabilidade externo.

5. **LLM injetável com cascata de fallback:** A LLM é passada via parâmetro (`run_assistant(query, llm=...)`) ao invés de ser instanciada dentro do grafo. No startup, o sistema tenta carregar o melhor modelo disponível em cascata:

   (a) **Fine-Tuned LoRA** (path local ou Hugging Face Hub) — requer CUDA;

   (b) **Fine-Tuned GGUF** (arquivo quantizado ∼4 GB) — funciona em CPU e Apple Silicon;

   (c) **Mistral 7B baseline** (Hugging Face Hub) — requer CUDA;

(d) **Sem LLM** — respostas baseadas apenas nos protocolos recuperados.

A detecção é automática: se `torch.cuda.is_available()` retorna `True`, carrega via `transformers`; caso contrário, tenta GGUF via `llama-cpp-python`. Ao trocar de modelo, o anterior é liberado da memória (`del model` + `gc.collect()` + `torch.cuda.empty_cache()` para evitar dois modelos simultâneos na RAM/VRAM.

6. **Paciente antes de protocolo + busca híbrida:** Para queries tipo "ambos", o grafo busca os dados do paciente *antes* dos protocolos. Isso permite busca híbrida no ChromaDB: primeiro pelos **sintomas** da query (capturando protocolos de emergência como IAM quando há "dor no peito"), depois por **comorbidade** do paciente (cobertura de condições crônicas). A query é pré-processada com remoção de stop words e nome do paciente para maximizar a relevância semântica.

5.10 Interface Gradio

A interface do assistente (`app/main.py`) é construída com Gradio e expõe os seguintes componentes:

- **Consulta médica:** campo de texto livre para a pergunta do médico;
- **Paciente:** dropdown filtrável com os 80 pacientes do banco (preenchido via PostgreSQL ou fallback JSON);
- **Seletor de Modelo LLM:** controle Radio com as opções disponíveis na máquina atual (Fine-Tuned, GGUF, Baseline ou Sem LLM), com detecção automática de hardware;
- **Status do modelo:** campo que exibe qual modelo está ativo e mensagens de carregamento/troca;
- **Resposta:** saída formatada com resposta clínica, nível de confiança (alta/média/baixa) e fontes consultadas;
- **Log de auditoria:** accordion expansível com o JSON completo do trace de execução.

Ao trocar o modelo pelo seletor, o sistema descarrega o anterior da memória e carrega o novo sob demanda (*lazy loading*), garantindo que apenas um modelo ocupe memória por vez.

6

Segurança, Guardrails e Explicabilidade

A aplicação de IA generativa em contexto clínico exige camadas de proteção que vão além da acurácia do modelo. Este capítulo detalha os três pilares de confiabilidade implementados no assistente: **guardrails de segurança**, **explicabilidade** e **auditoria**.

6.1 Princípios de Segurança em IA Médica

O design de segurança do assistente segue três princípios fundamentais:

1. **Nunca substituir o médico:** O sistema é uma ferramenta de *apoio à decisão*, não um tomador de decisão. Toda resposta deve ser validada por um profissional de saúde.
2. **Falhar de forma segura:** Quando houver incerteza, o sistema prefere recusar ou alertar a arriscar uma informação potencialmente danosa. Uma resposta conservadora com baixa confiança é preferível a uma resposta confiante mas imprecisa.
3. **Transparência total:** Cada resposta deve ser rastreável — quais fontes foram consultadas, qual o nível de confiança, quais regras foram verificadas e se houve retries.

Estes princípios se materializam nos três subsistemas descritos a seguir.

6.2 Guardrails: Validação Automática de Respostas

O agente de guardrails (`src/flows/agents/guardrails.py`) aplica quatro regras de validação a cada resposta gerada pelo agente de raciocínio. A validação é **determinística** — baseada em regex e palavras-chave — o que garante previsibilidade e auditabilidade, sem depender de um segundo modelo de linguagem.

6.2.1 Regra 1: Proibição de Prescrição Direta

O assistente não pode prescrever medicamentos. Frases como "prescrevo Losartana 50mg" ou "tome 2 comprimidos de 8/8h" são detectadas e rejeitadas.

```
PRESCRIPTION_PATTERNS = [
    r"prescrev[oa]",
    r"tome\s+\d+",
    r"administr[ea]\s+\d+\s*mg",
    r"iniciar?\s+(com\s+)?\d+\s*mg",
    r"receita[r:]",
    r"usar?\s+\d+\s*(mg|ml|comprimido)",
]
```

Listing 6.1: Padrões regex para detecção de prescrição direta.

Quando detectado, o feedback instrui o modelo a reformular como *sugestão* e indicar validação médica. Por exemplo, "considerar o uso de Losartana 50mg, conforme protocolo, sujeito à avaliação do médico responsável".

6.2.2 Regra 2: Validação Humana Obrigatória

Toda resposta deve conter um **disclaimer** indicando necessidade de validação por profissional de saúde. A verificação busca palavras-chave como "validação", "profissional de saúde", "médico responsável" ou "avaliação clínica".

```
DISCLAIMER_KEYWORDS = [
    "validação", "validar", "profissional de saúde",
    "médico responsável", "avaliação clínica",
    "decisão clínica", "confirmar com",
]
```

Listing 6.2: Palavras-chave de disclaimer de validação humana.

Se ausente, o guardrails reprova a resposta com feedback solicitando a inclusão da ressalva.

6.2.3 Regra 3: Proibição de Diagnóstico Definitivo

O assistente não pode afirmar diagnósticos de forma categórica. Frases como "o paciente tem diabetes" ou "confirmo que é pneumonia" são detectadas e rejeitadas.

```
DEFINITIVE_DIAGNOSIS_PATTERNS = [
    r"o paciente tem\b",
    r"o diagnóstico é\b",
    r"trata-se de\b",
```

```
    r"confirmo que\b",
    r"certamente (é|tem)\b",
]
```

Listing 6.3: Padrões regex para detecção de diagnóstico definitivo.

O feedback orienta o uso de linguagem probabilística: “sugere”, “é compatível com”, “deve ser investigado”.

6.2.4 Regra 4: Consistência com Contexto

Uma verificação heurística avalia se a resposta é compatível com o volume de contexto disponível. Se protocolos e/ou dados do paciente foram recuperados mas a resposta tem menos de 50 caracteres, o guardrails considera que o modelo pode ter “alucinado” uma resposta genérica e solicita elaboração.

6.2.5 Execução Sequencial das Regras

As quatro regras são executadas sequencialmente. Se qualquer regra falha, a resposta é marcada como `reprovado` e todos os feedbacks são concatenados para fornecer ao modelo de raciocínio uma instrução completa de correção.

```
GUARDRAIL_CHECKS = [
    ("no_prescription", _check_no_prescription),
    ("human_validation", _check_human_validation),
    ("no_definitive_diagnosis",
        ↪ _check_no_definitive_diagnosis),
]

for check_name, check_fn in GUARDRAIL_CHECKS:
    passed, message = check_fn(draft_response)
    if not passed:
        all_passed = False
        feedback_parts.append(message)

# Consistencia com contexto (separada)
passed, message = _check_context_consistency(
    draft_response, protocols, patient_data
)
```

Listing 6.4: Pipeline de validação dos guardrails.

6.2.6 Mecanismo de Retry com Feedback

Conforme descrito na Seção 5.4 do Capítulo 5, respostas reprovadas são devolvidas ao agente de raciocínio com o feedback concatenado. O mecanismo permite até **2 retries**. Se o limite for atingido, a resposta segue para a explicabilidade com advertências adicionais — o sistema nunca bloqueia completamente o fluxo.

Decisão de Design: Por que não bloquear?

Um bloqueio total geraria frustração no uso clínico e poderia atrasar decisões urgentes. A abordagem de "seguir com warnings" permite que o médico receba a informação acompanhada de alertas claros, preservando a autonomia do profissional.

O feedback injetado no prompt de retry tem formato específico:

```
ATENCAO - SUA RESPOSTA ANTERIOR FOI REPROVADA PELO GUARDRAILS:
Motivo: Prescricao direta detectada: 'prescrevo'.
Reformule como sugestao e indique validacao medica.
| Resposta nao inclui disclaimer de validacao humana.
Adicione uma nota indicando que a resposta deve ser
validada por um profissional de saude.
Corrija os problemas apontados nesta nova resposta.
```

Listing 6.5: Feedback de retry injetado no prompt do raciocínio.

6.3 Explicabilidade

O agente de explicabilidade (`src/flows/agents/explicabilidade.py`) transforma o rascunho validado em uma resposta **transparente**, adicionando quatro componentes: citações, confiança, advertências e disclaimer.

6.3.1 Citações e Rastreabilidade

Toda resposta inclui a lista de fontes consultadas, extraídas automaticamente dos metadados dos protocolos recuperados pelo ChromaDB e dos dados do paciente. As fontes são **separadas por relevância**:

- **Fontes diretas** — diretamente relacionadas ao quadro do paciente:
  - Protocolos cujo conteúdo contém termos das comorbidades: `[Pathway-Diabetes, Seção: Tratamento]`
  - Dados do paciente: `[Prontuário: Maria Santos]`, `[Exame: HbA1C, Data: 2026-01-15]`

- **Fontes complementares** — informação geral que pode ser útil, mas sem relação direta com as comorbidades do paciente.

A deduplicação é feita via conjunto (`set`), garantindo que cada citação apareça apenas uma vez mesmo quando múltiplos chunks vêm do mesmo documento.

6.3.2 Avaliação de Confiança

A confiança é calculada por um sistema de pontuação que considera a **quantidade** e **relevância** do contexto recuperado, a **disponibilidade** de dados do paciente e eventuais **penalizações** por retries:

Tabela 6.1: Sistema de pontuação para avaliação de confiança.

Fator	Pontos	Justificativa
3+ protocolos recuperados	$+3$	Múltiplas fontes convergentes
1–2 protocolos recuperados	$+2$	Ao menos uma fonte de evidência
Distância média < 0.5	$+2$	Alta relevância semântica
Distância média < 1.0	$+1$	Relevância moderada
Dados do paciente disponíveis	$+2$	Contexto individualizado
Cada retry do guardrails	-1	Indica problemas na geração

A classificação final segue três faixas:

- **Alta** (≥ 5 pontos): múltiplas fontes relevantes e contexto completo;
- **Média** (≥ 3 pontos): alguma fundamentação disponível;
- **Baixa** (< 3 pontos): informação insuficiente para alta confiança.

6.3.3 Advertências Contextuais

O sistema gera advertências automáticas quando detecta situações que merecem atenção:

- **Confiança baixa:** "Poucos dados disponíveis para fundamentar a resposta.";
- **Sem protocolos:** "Nenhum protocolo clínico foi encontrado para esta consulta.";
- **Retries realizados:** "Resposta foi refinada N vezes pelo guardrails de segurança."

6.3.4 Disclaimer Obrigatório

Independentemente do conteúdo, toda resposta inclui o disclaimer:

Disclaimer do Assistente

Esta resposta é uma sugestão baseada em protocolos clínicos e dados disponíveis. Toda decisão clínica deve ser validada pelo médico responsável.

6.3.5 Formato da Resposta Final

A resposta final apresentada ao médico segue uma estrutura padronizada:

```
[Resposta clinica do agente de raciocinio]

Fontes consultadas:
  . [Pathway-Diabetes, Secao: Tratamento]
  . [Pathway-Hipertensao, Secao: Exames de Seguimento]
  . [Exame: HbA1C, Data: 2026-01-15]

Fontes complementares:
  . [Pathway-PacienteCirurgico, Secao: Exames Complementares]

Confianca: Alta

Advertencias:
  . Resposta foi refinada 1x pelo guardrails de seguranca.

Esta resposta e uma sugestao baseada em protocolos clinicos
e dados disponiveis. Toda decisao clinica deve ser validada
pelo medico responsavel.
```

Listing 6.6: Formato da resposta final com explicabilidade.

6.4 Auditoria e Logging

O agente de logger (`src/flows/agents/logger_agent.py`) é o nó final em todos os fluxos do grafo. Sua responsabilidade é **consolidar** o estado completo da execução em um registro estruturado e **persistir** para consulta posterior.

6.4.1 Estrutura do Registro de Auditoria

Cada interação gera um registro JSON com seis seções:

Tabela 6.2: Seções do registro de auditoria.

Seção	Conteúdo
`session`	Query original e ID do paciente
`triagem`	Tipo de query classificado e entidades extraídas
`context`	Quantidade de protocolos, fontes consultadas, disponibilidade de dados
`response`	Tamanho do rascunho vs. final, confiança, fontes citadas, advertências
`guardrails`	Resultado (aprovado/reprovado), feedback, número de retries
`agent_trace`	Lista cronológica de todos os agentes executados com timestamps

A seção `agent_trace` é construída de forma **acumulativa**: cada agente adiciona sua entrada ao campo `audit_log` do estado. Ao final, o logger consolida todas as entradas em um registro único, criando um **trace completo** da execução.

6.4.2 Persistência em JSONL

Os registros são persistidos em arquivos diários no formato **JSONL** (JSON Lines):

```
logs/
  audit_2026-03-06.jsonl    # Um registro JSON por linha
  audit_2026-03-07.jsonl
  ...
```

Listing 6.7: Estrutura de persistência dos logs de auditoria.

O formato JSONL foi escolhido por três razões:

1. **Append-only:** Novos registros são adicionados ao final do arquivo sem reescrever os anteriores, minimizando risco de corrupção.
2. **Streaming:** Cada linha é um JSON válido independente, facilitando processamento incremental com ferramentas como `jq` ou ingestão em sistemas de monitoramento.
3. **Simplicidade:** Não requer infraestrutura adicional (banco de dados, message broker), adequado para o escopo do projeto.

6.4.3 Rastreabilidade End-to-End

Com o registro de auditoria, é possível reconstruir o caminho completo de qualquer consulta:

1. Qual foi a query original e como foi classificada;
2. Quais protocolos foram recuperados e de quais fontes;

3. Se e quais dados do paciente foram consultados;
4. Qual foi o rascunho gerado pela LLM;
5. Se o guardrails aprovou, reprovou ou realizou retries;
6. Qual o nível de confiança e quais advertências foram emitidas;
7. O timestamp de cada etapa para análise de latência.

Esta rastreabilidade é essencial para:

- **Depuração:** Identificar por que uma resposta foi inadequada;
- **Compliance:** Demonstrar que o sistema seguiu todas as regras de segurança;
- **Melhoria contínua:** Analisar padrões de reprovação para refinar regras ou prompts.

6.5 Tratamento de Queries Fora de Escopo

Quando a triagem classifica uma consulta como `fora_de_escopo`, o fluxo é encurtado: a query vai direto ao logger, que gera uma resposta padrão educada:

Resposta para Query Fora de Escopo

"Desculpe, esta pergunta está fora do escopo do assistente médico. Posso ajudar com consultas sobre protocolos clínicos, dados de pacientes, tratamentos e condutas médicas."

O registro de auditoria é gerado normalmente, permitindo rastrear quais tipos de perguntas estão fora do escopo — informação útil para expandir a cobertura do sistema no futuro.

6.6 Limitações e Trabalhos Futuros

As implementações atuais de segurança, embora funcionais, possuem limitações conhecidas:

1. **Guardrails baseados em regex:** Padrões fixos podem gerar falsos positivos (ex: "o paciente tem histórico de…" é flagrado pela regra de diagnóstico definitivo) ou falsos negativos (prescrições reformuladas de forma indireta). Uma evolução natural seria utilizar um classificador baseado em LLM como segunda camada de validação.
2. **Confiança heurística:** O sistema de pontuação é determinístico e não considera a *qualidade* do conteúdo dos protocolos, apenas quantidade e proximidade semântica. Métricas de faithfulness (fidelidade ao contexto) poderiam enriquecer a avaliação.

3. **Logging local:** A persistência em JSONL local é adequada para demonstração, mas em produção seria necessário integrar com sistemas centralizados de observabilidade (ex: ELK Stack, Datadog, OpenTelemetry).

4. **Consentimento e privacidade:** O sistema registra a query completa no log de auditoria. Em ambiente de produção com dados reais, seria necessário implementar anonimização automática e políticas de retenção de dados conforme LGPD.